Axona

A Learning-Adaptive Distributed Hash Table

with Axonal Publish-Subscribe

Research brief · 25 K-node benchmark

David A Smith · Axona.net
davidasmith@gmail.com

Source, data, and simulator: `github.com/axona-net/dht-sim`
Live network: `axona.net` · signaling: `bridge.axona.net`

Why?

> "*Those who would give up essential Liberty, to purchase a little temporary Safety, deserve neither Liberty nor Safety.*" — Benjamin Franklin

Privacy is the precondition for free expression and association. A communication channel that always passes through a trusted intermediary leaks the *fact* of the conversation even when the contents are encrypted. Knowing *who talked to whom, when, from where* is, on its own, a powerful surveillance signal — used commercially for advertising and behavioral inference, used institutionally for law enforcement and intelligence.

End-to-end encryption is not enough. Encryption protects message contents but not metadata. As long as the routing fabric belongs to a single party — a server, a federation, an ISP — that party retains visibility into the network's *shape*, and that visibility is itself the asset.

A peer-to-peer routing fabric removes the custodian. No server, no privileged peer, no trusted coordinator: the routing layer itself becomes a participant-symmetric primitive in which no one party has a privileged view of the traffic.

The DHT is the minimum viable substrate for that property. Without it, every "decentralized" service is one trusted server away from being centralized again.

What is a DHT? Why we need them

A **distributed hash table** is the foundation for finding *anything* in a decentralized network: any node, given a key, can locate the value in O(log N) hops with **no central authority, no privileged peer, no trusted coordinator**.

Already in production at scale:

System	Year	Built on
BitTorrent Mainline DHT	2005+	Kademlia variant — millions of simultaneous nodes
Ethereum devp2p	2015+	Modified Kademlia — peer discovery for blockchain consensus
IPFS / libp2p	2015+	Kademlia + content routing — CID → provider mapping
Tor v3 hidden services	2017+	HSDir DHT — onion-service descriptor lookup
Coral DSHT	2004	Latency-aware proximity clusters — production CDN 2004 – ~2015
S/Kademlia	2007	Security-hardened Kademlia (sibling broadcast, disjoint paths)

Why a next generation now:

The systems above were designed for one workload at a time — file-sharing, blockchain peer discovery, content addressing — on networks of stable nodes. **Axona targets a different deployment**: heterogeneous device classes (browser to server), high churn, integrated pub/sub, and locality awareness as a first-class property of routing rather than a layered afterthought.

*The next-generation DHT is not "Kademlia plus locality." It is a routing fabric that **adapts** — that learns from the traffic it carries and survives the network it actually lives on.*

Long-Term Potentiation — the biology behind the design

The brain and a DHT face the **same engineering problem**: a node with a fixed budget of connections, a flood of competing signals, and a need to remember which paths actually carry useful traffic. Evolution solved it once. We borrow the solution.

Long-Term Potentiation (LTP) is the canonical mechanism for synaptic learning, discovered by Bliss & Lømo in rabbit hippocampus (1973). When two neurons fire together repeatedly across the same synapse, that synapse **persistently strengthens** — change that outlasts the stimulus by hours, days, a lifetime.

The molecular cascade, simplified:

- **NMDA receptors** act as coincidence detectors — glutamate from the pre-synaptic neuron only opens them if the post-synaptic neuron is *already* depolarized. The signal is "I fired *and* you fired."
- Calcium influx triggers **AMPA receptor insertion** into the post-synaptic membrane, increasing sensitivity for next time.
- Within ~30 minutes the change is consolidated: gene transcription, new protein synthesis, sometimes new dendritic spines. The synapse is durably re-weighted.

The opposing process is LTD (Long-Term Depression): low-frequency stimulation *weakens* synapses. LTP and LTD together are the brain's way of *learning* — adjusting which routes among neurons are easy to traverse.

The intuitive summary, from Donald Hebb's *Organization of Behavior* (1949):

"Neurons that fire together, wire together."
*This is the **Hebbian rule** — and it's what every weight in every artificial neural network ultimately abstracts. Axona, a Neuromorphic DHT (N-DHT), applies it not to perception, but to routing.*

From neuron to routing table

The translation is direct, not metaphorical. The brain and a peer-to-peer overlay are both networks of capacity-limited nodes that must learn from traffic which edges to keep.

Neuroscience	Axona (NH-1)
Synapse — a connection between neurons	A peer entry in the routing table (`Synapse` class)
Synaptic weight — readiness to fire next time	`weight ∈ [0, 1]`
LTP — co-firing strengthens the connection	Successful lookup increments weights along the traversed edges (`_reinforceWave`)
LTD — disuse weakens	Time-based decay each tick (`weight *= decayGamma`)
NMDA coincidence detection — both ends must participate	Weight only increments when the synapse actually carried successful traffic, not on every probe
Synaptic tagging (Frey & Morris 1997) — recently potentiated synapses are protected from overwriting	NH-1's `inertiaDuration` epochs prevent eviction of recently reinforced synapses
Pruning — neurons with weak / unused connections lose them	`_addByVitality` evicts the lowest-vitality (`weight × recency`) edge to make room

Axona is the Hebbian rule applied to overlay routing. *Every "weight," every "synapse," every "potentiation" in this deck refers to a precise, measurable mechanism in the source code — see NeuronNode.js, Synapse.js, and the `_reinforceWave` / `_addByVitality` methods that implement LTP and pruning directly.*

Axona — how it differs

Traditional DHTs (Kademlia, Pastry, Tapestry) treat the routing table as a **static data structure**: fixed buckets, one rule for replacement, no awareness of the traffic flowing through. They were designed in 2001-2002 for stable nodes — and they do not adapt to the network they live in.

Axona is a Neuromorphic DHT (N-DHT): every peer is a *synapse* — a learnable edge with a weight that grows with successful traffic (LTP) and decays without it (LTD). Routing consults those weights. Eviction picks the least-vital edge. The table changes as the network changes.

	Traditional DHT	**Axona (N-DHT)**
Routing table	Fixed K-bucket per stratum	Weighted **synaptome** *(the per-node set of weighted outgoing edges)*
Edge state	Live / dead	Weight ∈ [0, 1], recency, locked-on-use
Routing decision	Greedy XOR	Action Potential (XOR × weight × latency)
Adapts to traffic?	No	Yes — Long-Term Potentiation (**LTP**), triadic closure, hop caching
Adapts to churn?	Lazy bucket refresh	Active dead-peer eviction + temperature reheat
Pub/sub	Layered on top	Integrated as axonal delivery trees
Global lookup at 50 K (× Dabek 3δ floor)	Kademlia **548 ms (2.65×)** — worsens with N	NX-17 **243 ms (1.18×)** — plateaus at the floor

The trade is "fixed and analytical" → "adaptive and empirical". The whitepaper-correctness of K-buckets gives way to *measured* behavior — which is why the rest of this presentation is about measurement.

Headline result. *On 50 K nodes, NX-17 sits within 36 ms of the analytical lower bound that Dabek et al. (NSDI 2004) proved for any recursive O(log N) DHT — `total ≈ 3δ` where δ is the median pairwise one-way round-trip time. Kademlia is 2× further from that floor. The full 3δ floor analysis appears later in the deck (the 3δ floor section).*

Pub/sub belongs in the network

The end-to-end argument (Saltzer, Reed, Clark, 1984): functions that can be implemented at the endpoints should be implemented at the endpoints. Put nothing in the network that doesn't *have* to be there.

Pub/sub routing has to be there. A subscriber and a publisher cannot, end-to-end, discover the multicast paths that connect them — only the network can. Treating pub/sub as a separate "application overlay" forces it to reproduce the routing layer with no actual visibility into it. The end-to-end principle applies to *meaning*; **delivery is a network function**.

A decentralized DHT is the right place for it. Every node is *both* an endpoint and a relay. The mechanism is content-blind: it routes bytes toward a topic, with no inspection and no control over what travels.

Consequences:

- **Everyone becomes a broadcaster** — peer-symmetric, no privileged position, no central authority.
- The network is **not smart**. It has no control over the information it carries.
- It is a **protocol and a platform** for relaying that information.
- It is **hyper-connected**, within the physical limits of a phone or a browser.

This is the architectural commitment Axona inherits from the start. The whole DHT exists to make this delivery primitive viable.

Why a simulator

The environment our system runs on is **the Internet**.

We have attempted to build as fair and realistic environment simulation as we could. The interface and services it provides to system components will be identical to those of an actual deployed system.

This matters because after launch there is no easy way to monitor, repair or improve the network we deployed. By design we *can't*. The only honest path is a simulated Internet large enough for us to have a reasonable expectation of *verisimilitude* to study.

And everyone can see it. Source, data, every CSV in this deck — all open.

"As close to the actual system as possible" does not mean good. It means: this is what we have right now. We will continue improving it.

Why do we do this? Because we *can't* track how information flows through the real system. We can't understand it — much less figure out how to repair it or improve it — without the ability to simulate it.

The lab bench

Purpose-built simulator · ~25 K lines of JavaScript · open-source.

Modeled with fidelity

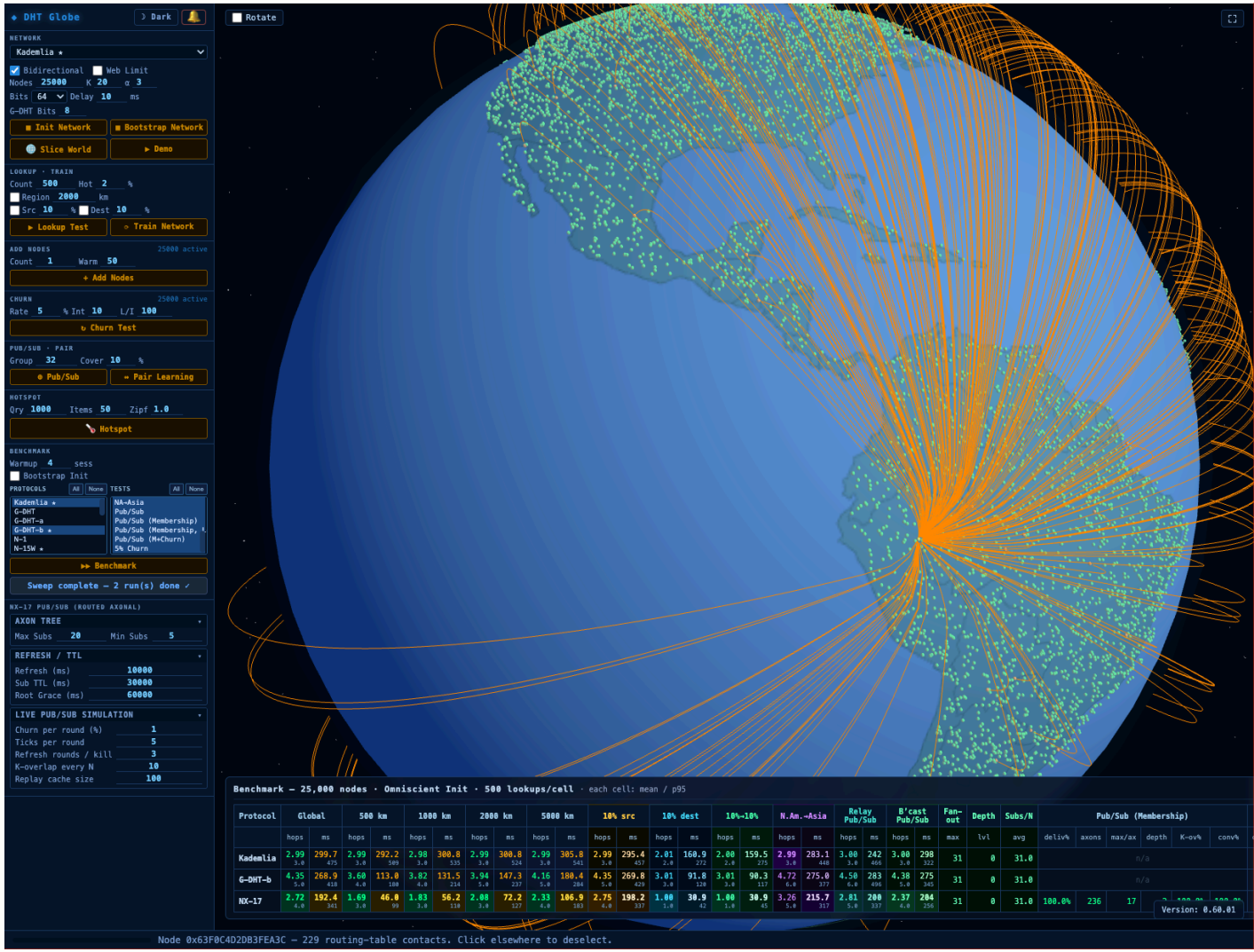
- GeoJSON land mask; haversine distances
- Up to 50 000 nodes on a navigable 3-D globe
- Per-hop simulated latency including 10 ms transit cost
- Bilateral connection caps mirroring WebRTC (Web Real-Time Communication) limits
- Every hop, ACK, reroute captured

Abstracted

- No wall-clock transport or encryption
- In-process node identity

Reproducibility

- All protocols build from the *same* seeded node set
- CSV export per run — every number in this deck is from `results/*.csv`



Simulator integrity

Five invariants govern every measurement in this deck: **(1)** the bilateral connection cap is honestly enforced (a base-class guard rail audits `connections.size ≤ maxConnections` at post-init, post-warmup, and after every churn round); **(2)** locality is preserved — no optimization reads another node's routing table directly; **(3)** RPC responses are bounded to k=20 peers, the envelope a real WebRTC node could actually return; **(4)** per-protocol parameters reach the protocol they were configured for, with no silent defaults; **(5)** the geographic prefix width is a runtime parameter that flows into every protocol's node-ID construction, including the `geoBits = 0` ablation runs.

Every invariant on this list has been violated at least once during development. *The deck shows post-fix measurements only. Skepticism is the methodology, not a hindrance.*

Glossary — vocabulary used across this deck

This work spans DHT engineering, reinforcement-learning ideas, and neuroscience. A reader expert in one domain may not know the others' vocabulary. **Every term has one and only one corresponding artifact in the source code** — the names are descriptive, not metaphorical.

Term	Domain	One-line definition
Synapse	Neuro → Axona	One directed *outgoing* routing edge with a learned weight ∈ [0, 1]
Synaptome	Neuro → Axona	The full set of outgoing synapses at a node — bounded at **50**
Neuron	Neuro → Axona	A node: synaptome + temperature + message handlers
Axon	Neuro → Axona	A directed delivery tree for one pub/sub topic, grown by routed subscribe
Vitality	Axona	The unified `weight × recency` score that drives every admission and eviction decision. See *FORGET*
AP score *(Action Potential)*	Axona	Learned-weight ranking on candidate next-hops — combines XOR progress, weight, and latency. See *NAVIGATE*
K-bucket	DHT (Kademlia)	Per-stratum routing-table slot, holding up to K=20 peers at a given XOR distance
XOR distance	DHT	The Kademlia metric `d(a,b) = a ⊕ b` — symmetric, halving-friendly
Stratum	DHT	One XOR-distance level. Stratum *b* = peers whose IDs first differ from yours at bit *b*
K-closest	DHT	The K nodes whose XOR distance to a target is smallest — used in routing and replication
Churn	DHT	Nodes joining and leaving; measured here as % of original network replaced
Sponsor-chain bootstrap	DHT	New node joins by routing through an existing peer; routing table fills from observed traffic
Anneal / annealing	RL	Gradually reduce randomness over time, transitioning from exploration to exploitation
Epsilon-greedy	RL	With probability ε take a random action; otherwise take the best-known — cheap insurance against premature lock-in

Capacity note. 50 is the cap on *outgoing* synapses. The total bilateral connection budget per node is **100** peers (≈ 50 outgoing + 50 inbound) — chosen as a safe cross-browser WebRTC target. Both numbers are pragmatic ceilings, not architectural commitments.

Protocols we will examine

We benchmark four DHTs at 25 K nodes under identical conditions. Each builds on its predecessor:

Protocol	What it adds	Inherits from
K-DHT (Kademlia, 2002)	XOR distance metric, K-buckets, α-parallel lookup	—
G-DHT (geographic, this work, 2025)	S2 cell prefix in node IDs ⇒ regional locality	K-DHT
NX-17 (predecessor state of the art, **SOTA**)	18 specialized rules, peak performance under tight cap	G-DHT *(via NX-1 … NX-15)*
Axona / NH-1 (this work, 2026)	Vitality-driven synaptome, unified admission gate	**NX-17** *(consolidation)*

Axona's current implementation, NH-1, is *not* a fresh parallel design — it is the result of a careful analysis of NX-17 and every protocol before it. Each NX-17 rule was studied for what it does, why it was added, and whether its work could be folded into a smaller surface area.

- **NX-17** carries the lineage: 18 specialized rules, 44 parameters, ~2300 lines.
- **NH-1** consolidates that lineage into the form Axona ships: 12 rules, 12 parameters, ~270 lines — every admission decision through a single vitality score.

*We selected NH-1 as the deployment target for Axona based on its **maintainability and understandability**. We continue to use NX-17 as the reference benchmark — the bar that NH-1 should approach, and that future work should match or surpass.*

K-DHT — Kademlia, the foundation

Kademlia (Maymounkov & Mazières, IPTPS 2002) is the routing substrate **every Axona node still uses at its bottom layer**. Full mechanism + Axona comparison appears later in the deck (*Kademlia vs Axona* in the comparison section).

Mechanism in one paragraph. Each node maintains **K=20 peers per XOR-distance bucket** — `d(a, b) = a ⊕ b` interpreted as an integer; symmetric, triangle-inequality. Lookup issues **α=3 parallel asynchronous `FIND_NODE` queries** to the closest known peers and iterates until the K closest are reachable. Bucket eviction is LRU-with-old-bias: live old contacts are kept (Saroiu observation: longer uptime predicts longer uptime), only stale ones replaced. Production: BitTorrent Mainline DHT, Ethereum devp2p, IPFS / libp2p, Tor v3 hidden services.

Why $\log_2$ N hops, exactly. Kademlia's XOR metric over an *n*-bit identifier space has ***n* buckets, each covering a distance interval** `[2^i, 2^(i+1))`. Every lookup hop is *guaranteed* to find a node at least one bit closer to the target than the current node — equivalently, **the remaining XOR distance halves on every hop**. Halving the search space = $\log_2$. With 160-bit IDs and *N* live nodes, the expected hop count is `[log₂ N]` — about **20 hops at 1 M nodes**, **17 at 100 K**, **14 at 10 K**.

Why DHTs are slow. Random IDs spread peers homogeneously across the globe; an average pair sits roughly **half the planet** apart — about **100 ms one-way RTT**. The naïve calculation:

> **Average message time ≈ 20 hops × 100 ms ≈ 2 seconds (at 1 M nodes).**

Too slow for real-time applications. *This is the gap Axona exists to close.*

Limit	Evidence at 25 K
No locality awareness	500 km lookup = **510 ms** — identical to its 499 ms global lookup
Fixed buckets	Same K peers regardless of usefulness; no response to traffic
Lazy churn repair	Broken edges persist until next bucket refresh
Broadcast cost O(audience)	Each pub/sub recipient reached by an independent lookup

The data structure is frozen. The network is not. K-buckets were a 2002 answer to "what's a stable routing table?" — Axona keeps the structure and replaces

G-DHT — adding geographic locality

The change: `nodeId = S2 cell prefix (8 bits) ‖ H(publicKey)`. XOR in the ID space now approximates physical distance — same K-bucket routing as Kademlia, no other code changes. *(More on the S2 cell prefix on the next slide.)*

Why this dramatically improves performance. Most of a user's connections are *geographically local* — social, transactional, real-time. With a location prefix, **local connections are XOR-close in ID space** as well as physically close, so the metric becomes meaningful for the traffic that actually dominates.

Back-of-the-envelope. With ~**10 K nodes in a region** and **~7 ms per hop** within a continental cell: $\log_2(10K) \approx 13$ hops × 7 ms ≈ **90 ms** for an in-region message. Compare against Kademlia's ~2 s for the same logical operation on a 1 M-node network.

Network of networks. When communicating with a far-away node, G-DHT first finds *any* peer inside the target's geographic neighborhood; once there, the remaining hops are short and efficient. The architecture is a hierarchy: long-distance edge → short-distance refinement. **This also improves global performance** — long-haul hops happen *once* per lookup, not at every step.

Measured result at 25 K (our simulator):

- 500 km regional latency: 510 ms → **150 ms** (3.4× faster)
- Global latency: 498 ms → **287 ms** (1.7× faster)

The tradeoff: security vs performance. A pure-Kademlia public-key ID is a random number — its position in the network reveals nothing about the node holder. A geographic prefix lets an attacker surmise the *approximate region* of the node. The risk is offset by the sheer number of nodes per cell, and a node holder may also spoof their location for added privacy at the cost of routing efficiency. *Failure modes detailed two slides on (S2 — security implications).*

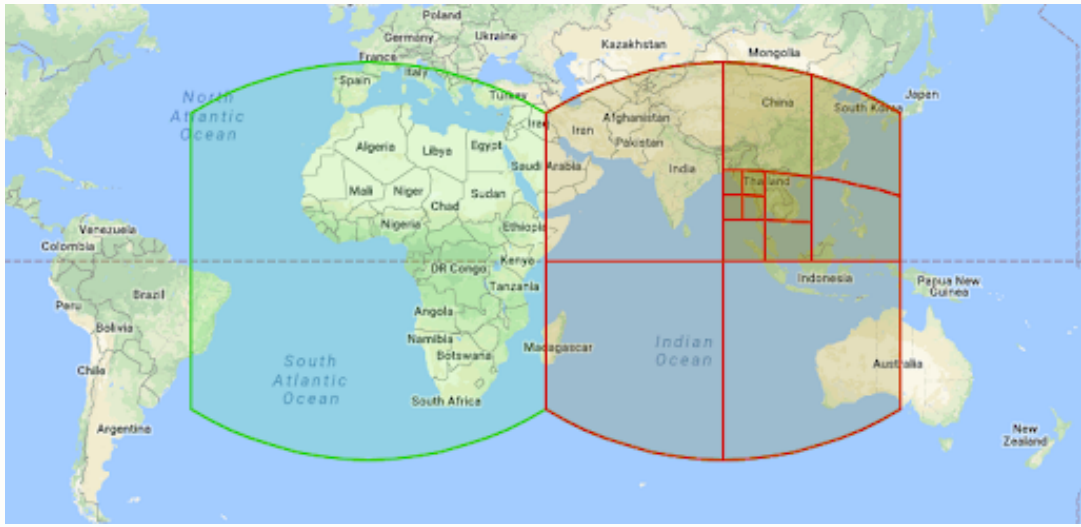
But G-DHT is still a *static* routing algorithm — no learning, no dynamics. The prefix is a one-time topology decision; pub/sub is still bolted on via K-closest replication, which drifts under churn.

The S2 library — what the cell prefix actually is

S2 (Google, 2011) is a hierarchical decomposition of the sphere onto a Hilbert space-filling curve, projected through six cube faces. Every point on Earth maps to a 64-bit cell ID; every prefix length defines a successively coarser tile.

- **Top 8 bits** — 3 bits encode the cube face (values 0 – 5; six faces, two unused encodings) and the next 5 bits subdivide that face along the Hilbert curve. **6 × 32 = 192 tiles** worldwide, each ≈ **continent-scale** (e.g. "western North America", "South-East Asia"). This is what we embed in every node ID.
- **Hilbert curve property** — geographically adjacent points have numerically close cell IDs. XOR distance in ID space ≈ physical distance, *for free*.
- **Sub-cell hierarchy** — refining the prefix bit-by-bit subdivides the tile in half along the Hilbert curve. 30 bits ≈ city block; 40+ bits ≈ metres.

*S2 gives us **locality for free** — at the cost of trusting that nodes don't lie about where they are.*



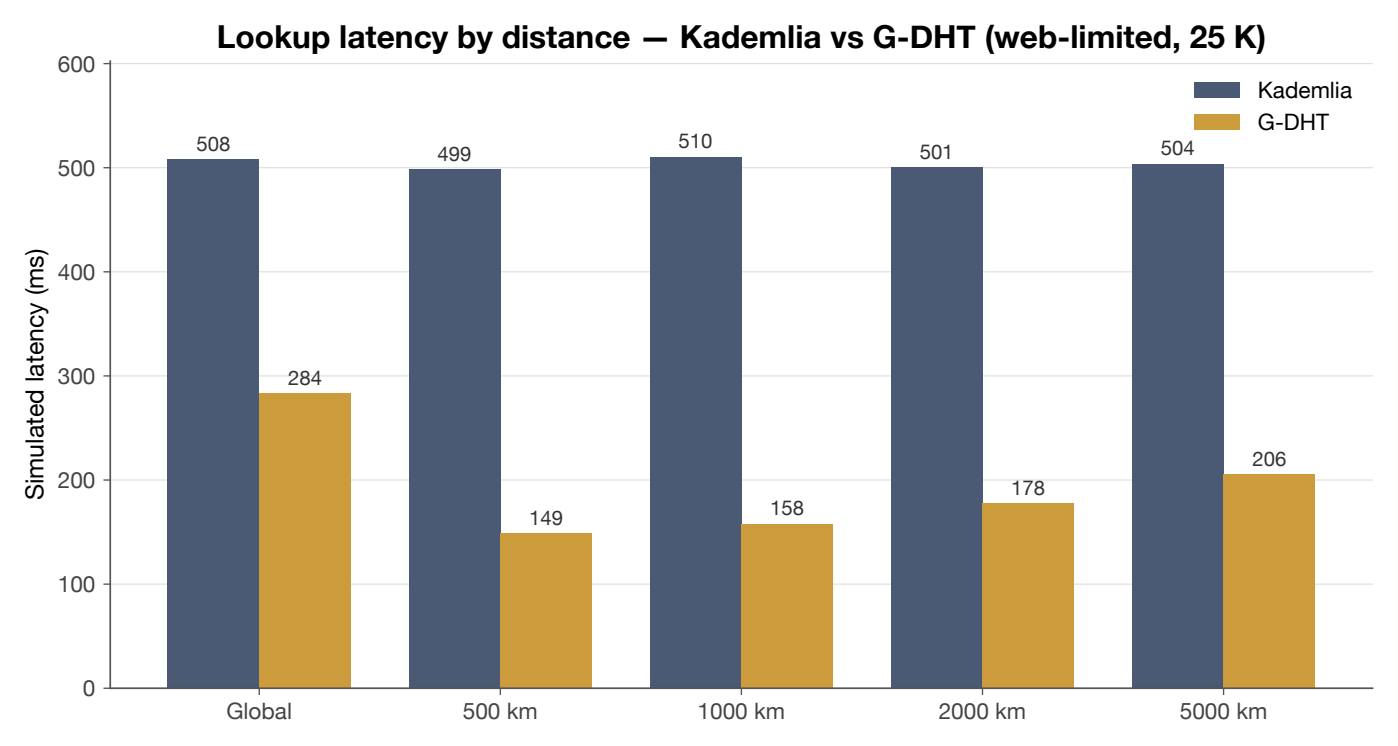
S2 — security implications

The S2 prefix in a node ID is **self-declared**. A node can claim any prefix it wants. This has three consequences:

- **The S2 prefix is not a trust primitive.** Never use it for authorisation, regional permissions, or anything resembling a capability check.
- **Prefix-forgery is real.** A malicious actor can pick a prefix to land in a different region. The benign failure mode is degraded routing (mis-located peers misroute traffic). The adversarial failure mode is a **Sybil swarm** in a target cell — many forged identities clustering on one region's address space.
- **Proof-of-location is the obvious defense.** Verifiable round-trip time (**RTT**) triangulation, GPS attestation, or trusted-witness schemes could anchor a claimed prefix to a measurable physical reality. This remains future work.

The honest framing: today's locality is a *cooperative* primitive. It works because well-behaved peers don't lie. The protocol does not depend on the prefix being honest, but its locality benefits do — and an attacker can degrade them without affecting correctness.

G-DHT — locality at a glance



Lookup latency by distance, web-limited 25 K. G-DHT's geographic prefix gives regional traffic a 3× speedup over Kademlia. Global lookups still pay the full hop cost.

Axona / NH-1 — our chosen design

The change: every peer becomes a *synapse* with a learnable weight. The routing table is no longer fixed K-buckets — it is a **synaptome** that adapts to traffic, prunes by vitality, and grows via learning rules (LTP, triadic closure, hop caching).

The architectural arc:

- **Inherits** the entire NX lineage (NX-1 → NX-17): K-DHT's XOR metric, G-DHT's geographic prefix, NX-17's AP routing, LTP, triadic closure, hop caching, axonal pub/sub
- **Consolidates** five separate admission mechanisms (stratified eviction, two-tier highway, stratum floors, synaptome floors, adaptive decay) into a single vitality gate
- **Removes** rules whose contribution didn't measurably justify their parameter footprint
- **Keeps** every behavioral surface that NX-17 measurably needed

Design priority: simplicity.

	NX-17	NH-1
Rules	18	**12**
Parameters	44	**12**
Lines of code	~2300	**~270**
Admission gate	5 mechanisms (stratified eviction, two-tier, stratum floors, synaptome floors, adaptive decay)	**1** (`_addByVitality`)

We accept a small performance gap against NX-17 in exchange for an architecture that one engineer can hold in their head — and that future engineers can extend without archeology.

Lineage note. Axona's synaptome **consolidates Pastry's three-tier routing state** (Rowstron & Druschel 2001) into a single vitality-scored set: high-vitality entries play the role of Pastry's **leaf set** (terminal-hop core), mid-vitality entries play the role of the **routing table R** (XOR-stratum coverage), low-vitality and 2-hop-sample entries play the role of the **neighborhood set M** (annealing replacement pool). The roles persist; the bookkeeping is unified.

NX-17 — the reference

Two research generations led here.

Launch pad: N-1 → N-15W. The original neuromorphic DHT family — fifteen-plus variants — explored the design space: synapses with weights, activation potentials, simulated annealing, hop caching, lateral spread, dendritic relay trees. Each pushed on one mechanism and exposed two new failure modes. By N-15W we had a working but tangled protocol with too many entangled rules to refactor in place.

Focused iteration: NX-1 → NX-17. The NX series re-grounded the architecture from a clean base. Each generation (NX-1, NX-2W, NX-3, … NX-17) addressed *one* concrete failure observed in its predecessor — one rule at a time, each measured against a benchmark. **NX-17 is the result.**

NX-17 is *better* than NH-1 under tight connection caps — by a small but real margin (~9 % global ms, ~18 % regional ms; head-to-head numbers in the *NH-1 vs NX-17* slide later in the deck).

It carries 18 specialized rules, 44 parameters, and ~2300 lines. Each rule earned its keep against a measured failure mode. Together they form a system hard to understand and hard to extend.

*We use NX-17 as the **reference benchmark**. The full rule list — and NH-1's disposition for each — appears in the NX-17 → NH-1 disposition table later in the deck.*

Axona in one slide — five operations, one vitality model

Axona's NH-1 implementation collapses the entire protocol into **five operations**, each scored by a unified vitality function.

Operation	What it does
NAVIGATE	Action Potential (**AP**) routing + 2-hop lookahead + iterative fallback
LEARN	LTP, hop caching, triadic closure, incoming promotion
FORGET	Continuous decay + vitality-based eviction
EXPLORE	Temperature annealing + epsilon-greedy first hop
STRUCTURE	Stratified bootstrap + mixed-capacity (highway) deployment

```
vitality(syn) = weight × recency
```

A single `_addByVitality()` admission gate replaces stratified eviction, two-tier highway management, stratum floors, synaptome floors, and adaptive decay. **~12 parameters** end-to-end.

What we call vitality

Axona's NH-1 admits and evicts every synapse via one scalar:

```
vitality(syn) = weight × recency(syn)
```

weight ∈ [0, 1] — trained by Long-Term Potentiation; reinforced on successful routing paths; decayed each tick by `γ = 0.995`.

recency = `exp(−Δepoch / RECENCY_HALF_LIFE)`. Two parameters control the time scale, both in NH-1's 12-parameter budget:

- **`INERTIA_DURATION = 20` epochs** — after a reinforcement, recency is locked to 1.0 for 20 lookups. This is the LTP protection window: a freshly used synapse cannot be evicted, regardless of vitality competition. Below this, learning would be self-destructive — the system would discard the very edges it just discovered to be useful.
- **`RECENCY_HALF_LIFE = 50` epochs** — once the inertia window expires, recency decays exponentially with a half-life of 50 lookups. After ~150 lookups with no reinforcement, recency is below 0.13 and the synapse is highly evictable.

The two factors are conventional individually. Hebbian potentiation (Hebb, 1949) gives the weight; exponential decay since last use (Ebbinghaus, 1885; LRU-K caching, O'Neil 1993) gives the recency. The closest biological analog is **synaptic tagging and capture** (Frey & Morris, *Nature* 1997) — synapses with both recent activity *and* sufficient potentiation are preferentially retained.

*The contribution of NH-1 is not the term or the formula. It is the use of this product as a **single admission gate** replacing five specialized mechanisms in NX-17: stratified eviction, two-tier highway management, stratum floors, synaptome floors, and adaptive decay.*

Axona / NH-1 rules — the full set, by operation

#	Rule	Operation	Why this rule
1	Stratified bootstrap	STRUCTURE	Cold-start coverage across all XOR distances
2	Mixed-capacity (highway%)	STRUCTURE	Realistic deployment — some peers run on real servers
3	AP routing	NAVIGATE	Learned-weight greedy walk dominates pure XOR
4	Two-hop lookahead	NAVIGATE	One probe of α second-hop options unblocks dead ends
5	Iterative fallback	NAVIGATE	If no progress, expand to k-closest from the synaptome
6	Long-Term Potentiation	LEARN	Reinforce edges on successful paths
7	Triadic closure	LEARN	If A→B→C succeeds twice, learn A→C
8	Hop caching + lateral spread	LEARN	Intermediate nodes cache the destination; the new edge is also propagated to the source's geographic neighbors
9	Incoming promotion	LEARN	Peers that contact me often become outbound synapses
10	Vitality-based eviction	FORGET	Drop the least-vital synapse when capacity is reached
11	Temperature annealing	EXPLORE	Cool exploration rate over time; reheat on dead-peer discovery
12	Epsilon-greedy first hop	EXPLORE	Small probability of random first hop avoids local minima

Every rule has a measured contribution at 25 K nodes. Ablations in Appendix C of the whitepaper.

STRUCTURE — bootstrap and deployment realism

Stratified bootstrap. Each new node is seeded with peers covering all XOR strata uniformly. Without this, the cold-start synaptome is dominated by lucky neighbors — local hops form, long hops don't. **The locality-preservation idea** — *route the join message through an existing nearby peer and fill your routing table from the nodes encountered along the way* — is **Pastry's locality-preserving join** (Rowstron & Druschel, Middleware 2001) carried over to Kademlia-style XOR strata.

Mixed-capacity deployment ("highway %"). Real P2P networks are heterogeneous: most peers are browsers (WebRTC ~100 connections), some are server-class (effectively unlimited).

A configurable `highwayPct` fraction of nodes is promoted to **server-class**: they accept unlimited inbound, hold a synaptome of up to 256, and act as transit hubs. The rest stay browser-class with the standard 50-synapse cap.

The deployment-realism implications — including the cheapest highway% that captures most of the available improvement — are quantified later in the *Highway %* section.

NAVIGATE — Action Potential routing

Each hop, score every candidate by a learned function:

```
AP(syn, target) = progress(syn, target) × syn.weight × ½^(latency_ms / 100)
```

- **progress** = XOR distance reduction toward target
- **weight** = LTP-reinforced [0, 1]
- **latency penalty** = exponential — fast peers preferred at all distances

Two-hop lookahead. When no first-hop is decisively best, probe for the best second-hop candidates and pick the path with the highest combined score.

Iterative fallback *(also known in the P2P literature as **surrogate routing** — Zhao et al. Tapestry, JSAC 2004).* If AP returns no candidate (every neighbor is "wrong direction"), fall back to k-closest-from-synaptome and retry. This rescues lookups that would otherwise stall in dead corridors. The mechanism is canonical; we adopt the established term to make the lineage clear.

AP routing is not greedy XOR with weights bolted on. The latency penalty makes nearby fast peers preferred over slightly-better-XOR distant ones — this is what makes the protocol latency-aware rather than purely distance-aware.

LEARN — four reinforcement mechanisms

LTP (Long-Term Potentiation). When a lookup succeeds, every synapse on the successful path gets a weight bump and an inertia lock. Locked synapses cannot be evicted.

Triadic closure. When a node X observes peer A repeatedly routing through it to peer C, X introduces A directly to C. A gains a new synapse to C; X is no longer needed as middleman on future A → C lookups. The name comes from social-network theory: the open triangle A — X — C is *closed* into a direct A — C edge.

Hop caching + lateral spread. Each intermediate node on a successful lookup adds the *destination* to its synaptome — and the new edge is also propagated laterally to the source's geographic neighbors. The path becomes shorter on the next lookup to the same region; nearby peers see the shortcut on their *first* lookup, not just after they generate one themselves.

Incoming promotion. When a peer reaches out to me repeatedly via incoming synapses, I promote it to a real outbound synapse — passive learning of who's interested in me.

Together these four mechanisms turn every successful lookup into structural learning: shorter paths, new direct edges, promoted incoming peers. The routing table is rewritten by the traffic flowing through it — not by a separate maintenance pass.

FORGET — the unified admission gate

`_addByVitality(node, newSyn)` is called for *every* synapse addition: bootstrap, LTP, triadic, promotion, hop caching, annealing.

```
1. If synaptome has room → add.
2. Otherwise, find the lowest-vitality non-locked synapse.
3. If new synapse's vitality > victim's → swap.
4. Else → refuse silently.
```

This single function replaces:

- NX-17's stratified eviction
- NX-17's two-tier (synaptome + highway) management
- NX-17's stratum floors
- NX-17's synaptome floors
- NX-17's adaptive decay parameters

Continuous decay (γ = 0.995/tick) erodes weight uniformly. Under-used synapses lose vitality and become eligible for replacement; well-used ones stay locked.

EXPLORE — temperature and epsilon

Temperature annealing. Each node carries a temperature `T ∈ [T_min, T_init]`. Cool by `T *= 0.9997` each lookup. Higher T → more probabilistic synapse selection (Boltzmann-style); lower T → greedy AP scoring.

Reheat on dead-peer discovery. When routing finds a dead peer, spike `T = max(T, 0.5)`. Accelerates exploration to repair damage.

Epsilon-greedy first hop. With probability ε = 0.05, replace the first AP-selected hop with a random synaptome member. Cheap insurance against early lock-in to a suboptimal corridor.

Both exploration mechanisms are biased toward learning rather than fully random — the floor is uniform sampling over the *current synaptome*, not over the network.

Exploration is bounded and targeted. Epsilon-greedy at the first hop costs at most one detour per lookup; annealing replaces only the lowest-vitality synapse. Neither mechanism risks the routing structure that LTP has already proven valuable.

What pub/sub on a DHT must do

Five requirements — the rest of this section is the answer to each.

#	Requirement	Why hard on a DHT	Axona's answer
1	**Reliable delivery** at steady state	A naïve broadcast = N independent lookups → $O(N^2)$ cost	Routed axonal tree, fan-out via direct sends
2	**Churn resilience**	K-closest sets drift; publisher/subscriber views diverge	Routed re-subscribe; tree heals on every refresh
3	**Deterministic routing**	Subscriber & publisher must agree on the same root without negotiation	Publisher-prefix topic ID — both derive it offline
4	**ID stability**	Topic identity can't change as the membership churns	`topicId = publisher.cellPrefix(8b) ‖ hash₅₆(name)` — fixed at publisher's S2 cell
5	**Recovery from missed messages**	Subscribers reconnecting want history, not just future messages	Bounded replay cache at every relay; replay piggy-backs on subscribe

Axona's pub/sub stack is a direct, point-by-point response to these five constraints. The next four slides show the mechanism for each.

How we got here — the K-closest → Axonal tree journey

The current pub/sub architecture is the fourth attempt. Each earlier attempt fixed a real failure mode and introduced a new one. The design that follows is what was left when we stopped adding parts.

The three failed approaches

- **NX-15 — K-closest replication.** Subscribe stores at each of K=5 nodes closest to `hash(topic)`; publish hits any one. Worked at zero churn. Failed under load: publisher and subscribers compute K-closest from *different positions* in the network, and their top-K sets drift apart under churn. Delivery collapsed to ~38 % at 25 % churn — a coordination bug, not a routing bug.
- **NX-16 — masked-distance fix.** Tried to decouple K-closest selection from synaptome expansion by masking the top 8 ID bits in the distance metric. Routing collapsed to ~40 % delivery *even at zero churn*. **Lesson, archived as a cautionary example: the distance metric used to select candidates must match the gradient used to expand them.**
- **NX-15-style replication, generally.** Replication on top of routing tries to *paper over* coordination drift. The next generation removed replication entirely and let routing carry the membership.

The four NX-17 fixes that work (and that Axona inherits)

1. **Publisher-prefix topic IDs** — `topicId = publisher.cellPrefix(8b) ‖ hash₅₆(name)`. Publisher and subscribers derive the same root deterministically. No negotiation, no drift.
2. **Terminal globality check** — when greedy routing thinks it has reached the topic, do one `findKClosest(topicId, 1)` to confirm no globally-closer peer exists. Without this, two subscribers can elect different roots.
3. **External-peer batch adoption on overflow** — when an axon hits its capacity, pick a *synaptome peer* (not an existing child) as the new sub-axon and ship the appropriate subscribers in one batch. Two invariants prevent runaway recursion.
4. **All-axon periodic re-subscribe** — every role re-issues its subscribe on a 10 s interval. The re-subscribe *is* the liveness check — no separate ping, no parent tracking.

The pattern. *Three approaches that each papered over a coordination problem at the application layer. One approach that pushed coordination back into the routing layer where the DHT could actually do the work. That's the architecture the next slides describe.*

Pub/sub on top — axonal trees

Why the name? In a biological neuron, the **axon** is the *output* projection — a single fibre that branches, branches again, and finally synapses onto many downstream targets. Information flows *outward* from one source to many recipients along this branching tree. That is exactly the shape of a healthy publisher-to-subscribers fan-out.

The analogy is structural, not poetic. A pub/sub topic in Axona is rooted at one node (the topic's "soma"). Direct subscribers attach to the root; when the root has too many children, it delegates a sub-axon (a "branch") that takes over a subset of the subscribers. The tree grows toward the population that wants the topic — just as a real axon grows toward its targets during development.

Axonal trees — how they work

Topic identity (deterministic). `topicId = publisher.cellPrefix(8b) ‖ hash56(event_name)`. Both publisher and every subscriber derive the same 64-bit ID with no negotiation. The tree's root pins into the publisher's S2 cell — naturally close to its audience.

Subscribe is a routed message toward `topicId`. The first live axon role encountered on the path *intercepts* and adds the new subscriber to its children. If the walk completes with no axon found, the terminal node opens a new role and becomes the **root**. Every subscribe message also carries the subscriber's `lastSeenTs` and triggers a replay (covered on the next slide).

Publish goes through the same route to `topicId`, lands at the root, and then **fans out**: the root sends to its direct children; each axon sub-role recursively forwards to its own children. One DHT lookup, then pure tree forwarding.

Branching on overflow (batch adoption). When an axon's direct-child count exceeds `maxDirectSubs`, it picks an existing peer in its synaptome as a new sub-axon, partitions its current children by XOR proximity to that new sub-axon, and hands off the relevant batch in one `ADOPT_SUBSCRIBERS` message. The tree branches in O(1) DHT operations.

Self-healing via re-subscribe. Every role re-issues its subscribe on a 10-second refresh interval. The walk lands on whichever live axon is closest to `topicId` *now*. Parent died? The re-subscribe attaches to a different live ancestor. Tree got reorganized? Invisible to the subscriber. The re-subscribe **is** the liveness check — there is no separate ping.

*100 % delivery baseline; **100 % recovered delivery** under 5 % churn at 25 K nodes.*

Lineage. This mechanism is structurally **SCRIBE** (Castro, Druschel, Kermarrec, Rowstron — JSAC 2002, layered on Pastry): routed subscribe + reverse-path forwarding multicast tree. Axona inherits the recipe and adds two refinements — *adaptive re-subscribe* as the liveness primitive (no separate ping) and *batch adoption on overflow* (axon trees grow in O(1) DHT operations rather than per-subscriber). Bayeux (Zhuang et al. 2001, on Tapestry) is the analogous mechanism in the OceanStore family.

Temporal pub/sub — subscribe is a request for *history*

A subscribe is not just "send me future messages on this topic". It is **"send me every message I haven't already seen"**. Each subscribe carries a `lastSeenTs` — the highest publish timestamp this subscriber has observed.

Every relay node keeps a bounded ring buffer. When an axon role receives a publish, it records `{ json, publishId, publishTs }` in a local cache (capacity ≈ 100 messages — tunable per topic).

On subscribe arrival, the relay filters its cache to `publishTs > lastSeenTs` and replays the missed messages as a single batched message before forwarding the subscribe upstream.

Why this matters under churn:

The decentralized axon tree means every re-publish node — not just the publisher — holds a copy of recent history. If a parent dies and a subscriber's re-subscribe lands on a different live relay, that new relay can fill the gap from *its own* cache. **Healing and replay are the same mechanism.** No central log, no separate recovery remote-procedure call (**RPC**), no "catch-up" protocol.

A subscribe message in Axona is simultaneously a liveness probe, a tree-attach request, and a request for missed history. Three jobs, one envelope — that's the axonal healing model.

Production refinements — three patches from real-network deployment

The textbook K-closest model passed the simulator. Live deployment of **Axona** surfaced three corner cases:

Lazy-axon promotion. Upstream protocol *dropped* publishes that landed at a K-closest node which didn't yet hold a role → publish-before-subscribe lost messages. **Fix:** any K-closest receiver promotes itself on first publish and seeds the replay cache. Empty roles GC after `rootGraceMs` (60 s) so cost stays bounded.

Self-replay (lastSeenTs override). When a node receives a publish-k it advances its `_lastSeenTsByTopic` at the *network* layer. If that same node later subscribes, the cache filter `m.publishTs > lastSeenTs` excludes everything — even though the messages sit in its own cache. **Fix:** when `subscriberId === self`, replay the full cache directly into the local handler, ignoring `lastSeenTs`.

Multi-hop sendDirect. K-closest assumes any peer can reach any other in one hop. In a real WebRTC mesh, browsers behind NATs may not have direct DataChannels with every other browser. `sendDirect` to an unbound target was silently dropping. **Fix:** transport-layer fallback — wrap the message in `__tunneled_direct__` and route hop-by-hop via the existing Axona walk. The bridge becomes a routing hop among many, not a privileged relay.

All three patches are upstream in `@axona/protocol`. None change wire format; mixed-version networks degrade gracefully.

100-peer stress test — axonal pub/sub at scale

Single-process N=100 stress (`smoke_stress_100.js`): all peers form a full mesh, no bridge process, three scenarios.

Scenario	Setup	Result
A	1 publisher → 99 subscribers, 1 topic, 3 messages	**99/99** full delivery
B	publish-before-subscribe — 5 messages, then 50 late subscribers	**50/50** receive all 5 via replay
C	5 publishers × 5 topics × 95 round-robin subscribers	**5/5 topics**, all subscribers receive

Per-peer participation histogram at N=100:

Role	Count
subscriber-only	62
subscriber + axon	33
publisher + subscriber	3
publisher + axon	1
all roles	1

Every peer plays at least one role. Axon-roles distribute across the network as the K-closest set varies per topic — no concentration, no single point of failure.

End-to-end tick

```
on lookup(target):
  hop = 0
  while hop < MAX_HOPS:
    candidates = synaptome ∪ incomingSynapses
    next       = AP_score(candidates, target)
    if next == null: next = iterative_fallback(target)
    if hop == 0 and rand() < ε: next = random(synaptome)

    record_transit(prev, next)         // LEARN: triadic
    promote_incoming_if_warranted()    // LEARN: incoming
    hop_cache_destination()            // LEARN: hop caching
    anneal_replace_lowest_vitality()   // EXPLORE
    hop++

on success: reinforce_path()           // LEARN: LTP
periodically: decay_all_weights()      // FORGET
```

Every operation cost is bounded: O(synaptome.size). At 50 synapses the per-hop compute is ~0.2 ms — small relative to the 10 ms transit cost.

A complete Axona hop fits in this pseudocode. Five operations interleave on every lookup; learning is a side-effect of routing, not a separate phase.

Methodology

- **500 lookups per test cell.** Global, regional radii (500 / 1k / 2k / 5k km), pub/sub
- **Same node geometry** across all four protocols — direct comparison, not three independent builds
- **Bootstrap init** for production realism: sponsor-chain join + warmup. Omniscient init shown separately as a theoretical ceiling.
- **Churn** induced discretely (instantaneous kill) and continuously (1 % every 5 ticks)
- **Connection cap** 100 (browser-class, web-limited model)

Data provenance. *The 4-way comparison and Highway% / Slice World tables come from 25 K-node CSVs captured at simulator v0.67–v0.68. The 3δ floor analysis later in the deck is from fresh v0.69.00 sweeps at 5 K / 25 K / 50 K. Each table self-discloses its source; the footer reflects the most recent simulator version.*

What training does — and doesn't do

A non-obvious result from running every NX variant under two starting conditions:

- **Omniscient init** — every node seeded with its theoretically optimal K-closest neighbors
- **Bootstrap init** — sponsor-chain join + 50 K warmup lookups (production realism)

Protocol	Omniscient hops	Bootstrap-trained hops	Gap	Bootstrap success%
N-1	2.22	5.10	+130 %	✗ 60 %
NX-3	3.64	4.33	+19 %	✗ 73 %
NX-6	3.61	4.43	+23 %	**100 %**
NX-10	3.67	4.29	+17 %	100 %
NX-15	3.47	4.30	+24 %	100 %
NX-17	**3.67**	**4.28**	**+17 %**	**100 %**

Three findings:

1. **A shared asymptote.** Every NX from NX-6 onward lands in a tight 4.28 – 4.43 hop band under bootstrap+training, *regardless* of where it started. Training settles the synaptome to a **traffic-driven asymptote** near 4.3 hops, essentially independent of initial conditions.
2. **NX-17 has the smallest hop gap and fast bootstrap-trained latency.** Bootstrap-trained NX-17 evaluates greedy candidates faster than its own omniscient configuration: pruned synaptomes are leaner. (Latency is left as hops × per-hop cost in this table — the 4-way comparison earlier in the deck reports the headline NX-17 latency at the 25 K, default-warmup configuration.)
3. **Training is compute-optimizing, not path-shortening.** The 50-synapse cap is the binding constraint. Training redistributes weight on the edges sponsor-chains gave you at join — *it does not discover new shorter edges sponsor-chains missed*.

*The lever for production routing quality is the **initial synaptome construction**, not the training algorithm. Bootstrap-trained NX-17 hits a 4.3-hop asymptote at 100 % success — competitive with its own omniscient ceiling.*

Numbers in this table are from a controlled multi-variant ablation (50 K warmup lookups). The 4-way comparison earlier in the deck reports NX-17 at default warmup conditions and is the canonical headline latency.

Four-way comparison at 25,000 nodes

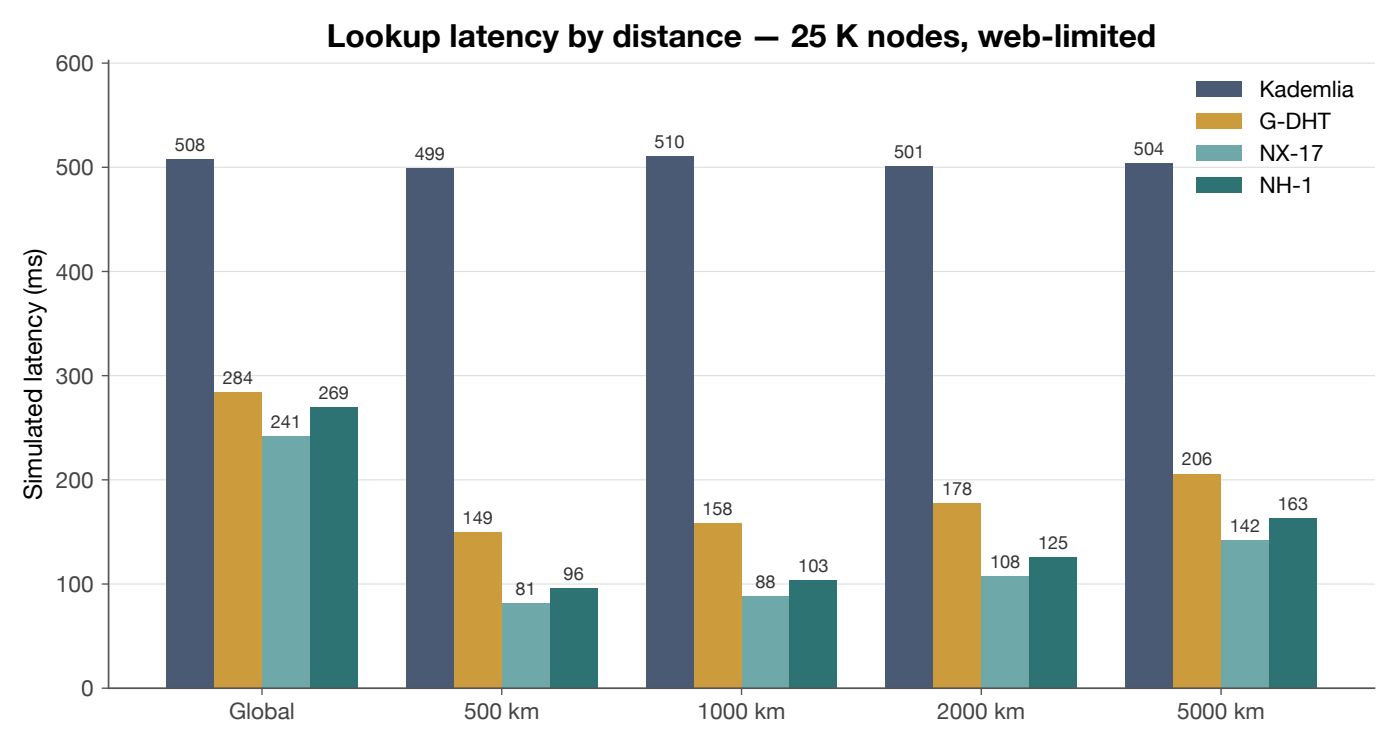
Web-limited (cap = 100), omniscient init, geoBits = 8, no highway promotion. Same node geometry across all four protocols. **Canonical init**: every protocol uses identical K-closest XOR-fill bootstrap so the routing/learning algorithm is measured in isolation from any per-protocol bootstrap strategy.

Test	Kademlia	G-DHT	NX-17	NH-1
Global (hops / ms)	4.53 / **508**	5.57 / 284	4.47 / **241**	5.26 / 269
500 km (hops / ms)	4.50 / 499	4.86 / 149	2.79 / **81**	3.30 / 96
1000 km (hops / ms)	4.58 / 510	5.04 / 158	2.97 / **88**	3.49 / 103
2000 km (hops / ms)	4.45 / 501	5.36 / 178	3.36 / **108**	3.96 / 125
5000 km (hops / ms)	4.49 / 504	5.44 / 206	3.74 / **142**	4.50 / 163
pubsubm delivered	n/a	n/a	**100 %**	**100 %**
pubsubm + 5 % churn (recovered)	n/a	n/a	**100 %**	**100 %**
dead-children / orphans	n/a	n/a	**0 / 0**	**0 / 0**

Both NX-17 and Axona's NH-1 dominate Kademlia and G-DHT on every distance band. NX-17 retains a small lead at the cap = 100 ceiling — see "Axona / NH-1 vs NX-17" later in this deck.

Native-init numbers (each protocol's own bootstrap strategy) sit within 5–10 ms of these on a per-cell basis; the 4-way ranking is identical. Canonical init is the headline measurement because it removes bootstrap variance from the algorithmic claim.

Four-way comparison — at a glance



Lookup latency (ms) per distance band, 25 K nodes web-limited. NX-17 (light teal) and NH-1 (deep teal) cluster well below Kademlia (slate) and G-DHT (amber) at every distance.

How fast is fast? — the 3δ floor

Latency comparisons need an absolute reference, not just "lower than Kademlia". **Dabek, Li, Sit, Robertson, Kaashoek & Morris** (MIT — NSDI 2004, *Designing a DHT for low latency and high throughput*) proved that for *any* recursive O(log N) DHT, total lookup latency converges to a hard analytical floor:

lookup ms ≈ δ + δ/2 + δ/4 + … = 3δ

where **δ** is the median pairwise *one-way* internet latency. Each successive lookup hop covers a geometrically halving fraction of the remaining ID-space, so the second-to-last hop is half the cost of the last, the third-to-last a quarter, and so on. The series sums to 3δ regardless of N. Even an oracle that always picked the lowest-RTT finger could not do better.

Measuring δ in our simulator. δ is a property of the network itself, not of any DHT — it depends only on (a) the population-weighted geographic placement of nodes and (b) our latency model `propagation = (haversine_km / 20015) × 150 ms + 10 ms` per one-way message. The same δ applies to Kademlia, G-DHT, every NX variant, and NH-1: none of them can route faster than 3δ.

At every benchmark sweep we sample **10 000 random node pairs** from the live population (independently of any protocol) and compute the one-way pairwise latency:

N	δ_median	δ_p95	3δ floor
5 K	67.9 ms	123.3 ms	**203.6 ms**
25 K	68.0 ms	124.1 ms	**204.0 ms**
50 K	68.9 ms	124.2 ms	**206.6 ms**

δ is stable across N — population character doesn't change with sample size. Coincidentally identical to Dabek's measured King-dataset δ = 67 ms, so the simulator's geometric latency model lands on the same point as real-world Internet RTT. The next slide then measures every protocol against this single shared reference line.

Any honest claim about DHT latency should be expressed as a multiple of 3δ. "Beats Kademlia" is a relative win; ***"close to 3δ" is an absolute one.***

Axona lives at the floor

Same global lookup test (500 random source/destination pairs). Each cell shows **measured median ms (× the 3δ floor for that N)**, so the reader sees both the absolute latency and how far above the theoretical lower bound it sits:

N (3δ floor)	Kademlia	G-DHT	NX-10	NX-17	NH-1
5 K (204 ms)	410 ms (2.01×)	268 ms (1.32×)	217 ms (1.07×)	**215 ms (1.06×)**	240 ms (1.18×)
25 K (204 ms)	503 ms (2.46×)	287 ms (1.41×)	243 ms (1.19×)	**241 ms (1.18×)**	254 ms (1.25×)
50 K (207 ms)	548 ms (2.65×)	291 ms (1.41×)	240 ms (1.16×)	**243 ms (1.18×)**	264 ms (1.28×)

- **Axona (and the broader N-DHT family) plateaus at ~1.18 × the floor** between 25 K and 50 K. NX-17 sits at **241 → 243 ms** — only ~36 ms above the 3δ lower bound.
- **Kademlia worsens with N** (2.01× → 2.65×, **410 → 548 ms**) — its log N hop tax compounds.
- **NH-1 trails NX-17 by ~10 %** (**~21 ms** at 50 K) — a real but recoverable cost of the 12-parameter simplification vs NX-17's 44.

The remaining 18 % at NX-17 has a clean structural explanation. NX-17 averages 4.5 hops where an oracle PNS-ideal lookup would take ~3. Each "extra" hop costs ~δ/2 ≈ 34 ms — exactly the geometric tail Dabek's series predicts.

Cross-reference: Tapestry RDP. The Berkeley Tapestry paper (Zhao et al., JSAC 2004) reports the same kind of result with a complementary metric — **RDP (Relative Delay Penalty)** = `route latency ÷ direct IP latency` — measured *per pair*, then summarized as median + 90th percentile. Their median wide-area RDP ≈ 1; 90th percentile ~2–3 (with location-pointer optimization). The 3δ floor is **median-only / theory-only**; RDP exposes per-pair tail behavior the median hides. Future work: report RDP-style distributions alongside the 3δ analysis to expose tail latency on bad source / destination pairings.

Implication. *Latency optimization within the O(log N) routing class is essentially complete. Further annealing / lookahead tweaks move 1.18× → maybe 1.10× at best — diminishing returns. The remaining R&D axes are churn resilience, pub/sub fan-out, and constant-hop variants.*

Learning beats locality — even without geography

A skeptical reader of the 3δ floor result might suspect the **S2 geographic prefix is doing all the work** — that the neuromorphic protocols are just sharpening routes that geography already establishes. To answer this directly, we strip the prefix (`geoBits = 0`, random IDs only, no geographic structure) and re-run.

Global lookup latency, 25 K nodes, canonical init *(every protocol starts from identical K-closest-XOR routing tables — controls bootstrap variance):*

Protocol	Without geo prefix (geoBits = 0)	With geo prefix (geoBits = 8)	Improvement vs Kademlia (no geo)
K-DHT (Kademlia)	**506 ms**	508 ms	— *(baseline; no geo by design)*
G-DHT	780 ms[1]	284 ms	— *(geo prefix is its only mechanism)*
NX-17	**376 ms**	**241 ms**	**–26 % from learning alone**
NH-1	467 ms	269 ms	**–8 % from learning alone**

The headline. With *no geographic information whatsoever*, **NX-17 is 26 % faster than Kademlia** and **NH-1 is 8 % faster** — purely from LTP-driven path reinforcement. Add the S2 prefix back in and the gap widens further (NX-17 → 1.18× the 3δ floor at 50 K). **Learning is doing real work**, not just sharpening pre-existing geographic structure.

Geography helps; geography is not necessary. *The neuromorphic learning algorithm delivers measurable latency benefit even on a topology with no exploitable structure — the cleanest possible "learning helps" claim. Detail and methodology in the geoBits = 0 — isolating the learning advantage slide later in the deck.*

[1] G-DHT's geoBits=0 result reflects its `noProgressLimit = 3` lookup-tuning choice (vs K-DHT's 2). At geoBits=0 with no real geographic gradient, the extra "no progress" rounds are wasted overhead. An architectural finding the methodology surfaced.

Highway %: deployment-realistic knee

What this slide measures. Real P2P networks are heterogeneous: most participants run inside a browser (~50–100 connections, the WebRTC ceiling), but some run on real servers with effectively unlimited inbound capacity. We sweep the *fraction* of server-class nodes from 0 → 100 % and measure Axona (NH-1) latency at each point.

What "highway" means. A **highway node** is a server-class peer: `maxConnections = ∞`, synaptome cap raised from 50 to 256. It accepts unlimited inbound and acts as a transit hub. The rest of the network stays browser-class.

Highway promotion within the simulator is random — no privileged coordinator picks them. In a real deployment, highway status is *self-determined*: a peer running as a node application on a PC or a server identifies itself as one based on its actual capacity. The protocol treats highway and browser nodes identically apart from their cap.

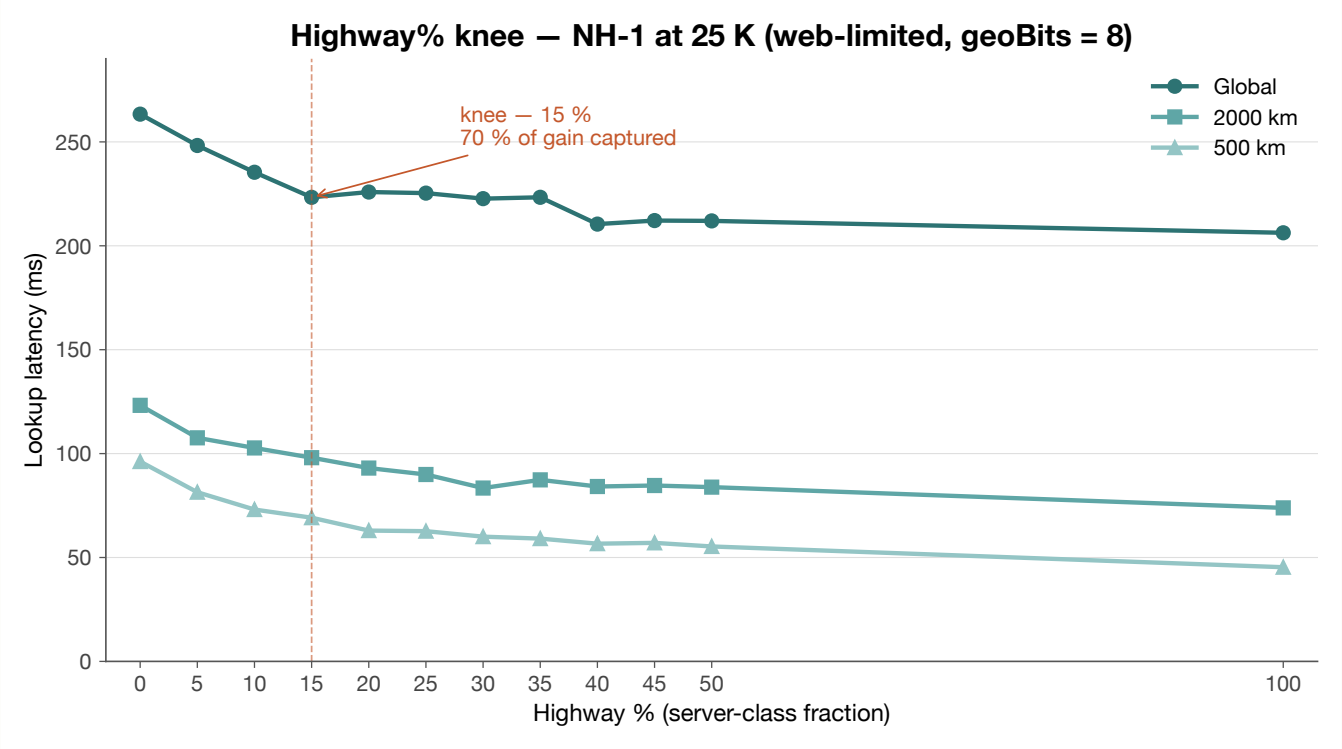
What the "knee" means. The point on the curve where additional server-class capacity stops paying off — adding more highway nodes beyond the knee yields rapidly diminishing latency improvement. It is the cheapest deployment that captures most of the available benefit.

Highway %	Global hops / ms	500 km ms	2000 km ms	Synaptome (highway)
0 (all browser)	5.09 / 263	96	123	n/a
5	4.61 / 248	81	108	217
15 (knee)	**4.16 / 223**	**69**	**98**	**218**
30	3.97 / 223	60	83	215
50	3.76 / 212	55	84	208
100 (all server)	3.52 / 206	45	74	244

15 % highway captures 70 % of available improvement *— a realistic deployment scenario where some peers run on powerful hardware.*

The hw = 0 row reads 5.09 / 263 ms; the headline 4-way comparison earlier in the deck reads NH-1 at 5.15 / 263 ms. Same configuration, two different runs — within run-to-run noise. Each table is consistent within itself.

Highway %: the knee, charted



Three latency curves (global, 2000 km, 500 km) across the highway% sweep. The knee at 15 % is highlighted. Beyond 50 %, the system is essentially saturated — it has the routing it can use.

Slice World — recovery from a network partition

The Slice World test partitions the network into Eastern and Western hemispheres connected through **a single bridge node** (placed near Hawaii). Every cross-hemisphere edge except those incident on the bridge is removed.

The question it asks is not "*can you find the bridge once?*" — it is "*given **one** intact connection across a severed network, can the protocol leverage that single hole in the dike into a flood of restored connectivity?*"

Protocol	Slice success%	Slice hops / ms	What happens
Kademlia	✗ **0.0 %**	—	Cannot reach the bridge. The partition is permanent — the network is now two networks.
G-DHT	✗ **4.6 %**	9.5 / 525	Same limitation; the geographic prefix doesn't help when no edge points at the bridge
NX-17	**94.8 %**	7.3 / 423	Bridge becomes the seed; learning amplifies it into recovered connectivity
NH-1	**94.4 %**	8.7 / 462	Same mechanism; consolidation costs nothing here

*A partition like this is what a real outage looks like — a submarine cable fault, an ISP-level filter, a regulatory split. The test measures whether the protocol can **heal** from it.*

Slice World — visualized

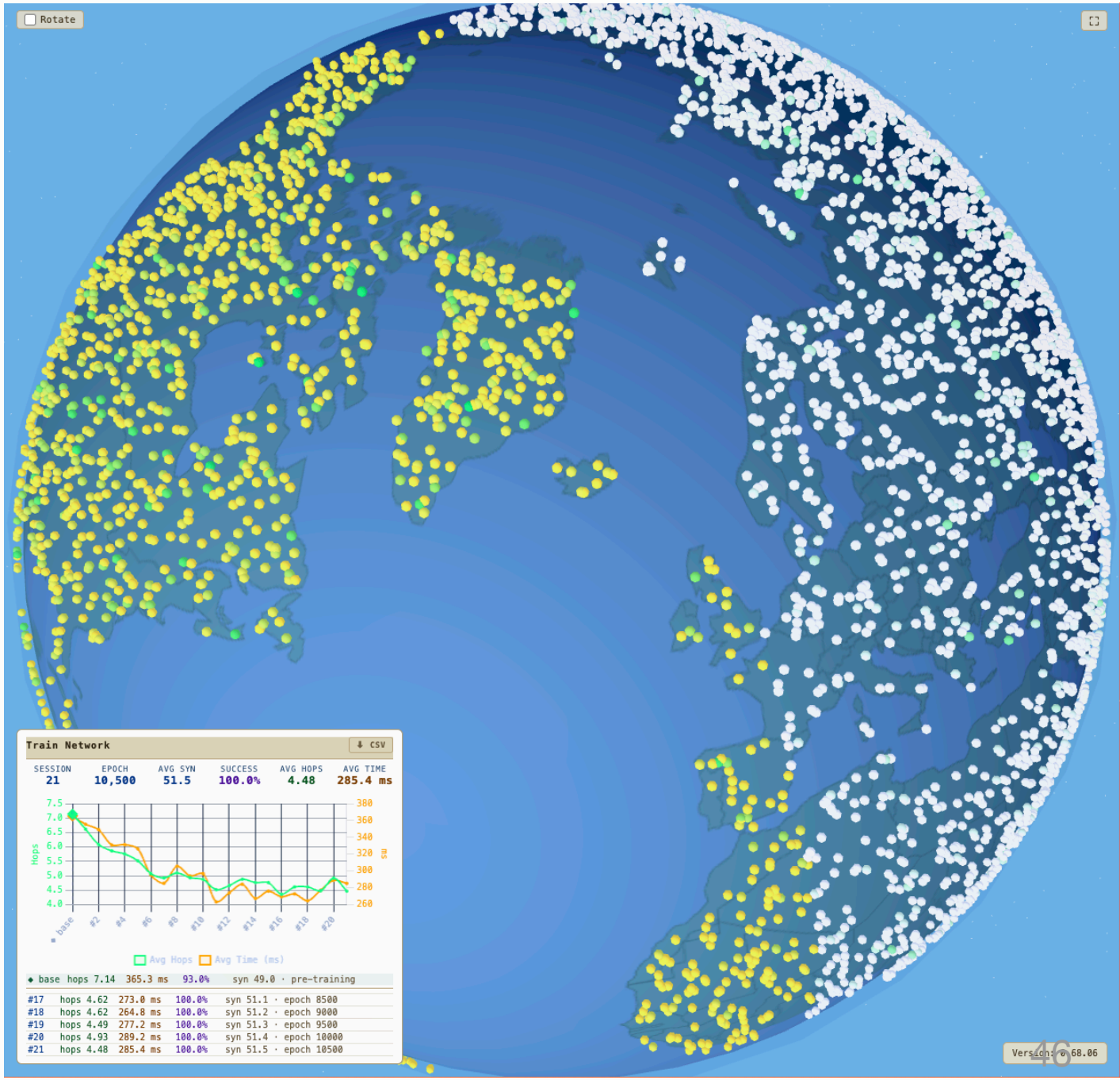
The simulator after Slice World setup at 5 K nodes.

Yellow = Western hemisphere
White = Eastern hemisphere
Green = at least one non-bridge cross-hem peer

At setup only **Hawaii** is green — it is the sole bridge between the partitioned halves.

With **Kademlia**, that picture stays frozen during training: no learning, no recovery.

With **NX-17** or **Axona**, green spreads outward as hop caching, triadic closure, and lateral spread deposit new cross-hem synapses on every successful path through the bridge.



Slice World — the bridge as a seed crystal

A diagnostic run shows the recovery happening directly. Starting from a freshly partitioned 5 K network — **zero cross-hemisphere synapses, partition honest** — and running just 10 lookups:

	value
Cross-hem synapses **before**	0
Cross-hem synapses **after 10 lookups**	**20**
Lookups that succeeded	7 / 10
Avg new cross-hem synapses per success	~3

Each successful bridge crossing seeds new connectivity. When a path goes `west-source → … → bridge → … → east-target`, Axona's learning rules fire on every node along the way:

- `_hopCache` — every intermediate node adds the *destination* to its synaptome (a fresh cross-hem edge for every west node on the path)
- `_recordTransit` — observed `(prev → next)` pairs become triadic-closure candidates
- `lateralSpread` — propagates the new synapse to the source's geographic neighbors

By 500 lookups, the partition has effectively dissolved. Hundreds of cross-hem synapses now exist; only the earliest lookups depended exclusively on the bridge. The 95 % aggregate success rate is the *integral* of recovery, not a snapshot of bridge-finding.

This is what the test was designed to measure. *Single-shot bridge-finding is the opening move; learning-driven re-stitching is the play. Axona doesn't route through the partition — it dissolves it.*

Pub/sub robustness

Metric	Axona (NH-1) at 25 K
Baseline delivery	100.0 %
Immediate (post-kill, no refresh)	99.9 %
Recovered (after 3 refresh rounds)	**100.0 %**
K-overlap (publisher / subscriber views)	99.5 %
K-set stability (recovered, pub / sub)	95 % / 95 %
Dead-children	0
Orphans	0
Attached %	100 %

Same numbers across the entire highway% sweep — pub/sub robustness is independent of capacity asymmetry.

*The axonal tree heals through **routed re-subscribe**. There is no separate liveness ping, no parent tracking, no gossip.*

Pub/sub under sustained churn — graceful degradation, no cliff

A continuous live-simulation: 25 K nodes, 79 groups × 32 subscribers, **1 % of alive non-publisher nodes killed every 5 ticks**, three refresh passes between kills. Cumulative churn measured as fraction of original network replaced.

Cumulative churn	Delivered %	K-overlap	Axon roles
0 %	**100.0 %**	100 %	537
5 %	98.7 %	—	1,541
10 %	91.2 %	81 %	1,787
15 %	88.7 %	77 %	1,989
20 %	86.5 %	62 %	2,116
25 %	70.0 %	54 %	2,169
30 %	52.4 %	47 %	2,189
34 %	50.8 %	42 %	2,197

Three observations:

1. **No cliff.** Delivery degrades smoothly with churn; there is no breakdown threshold. The system finds a steady state where new subscriber recruitment matches loss.
2. **K-overlap predicts delivery 1:1.** The dominant residual failure isn't broken routing — it's subscribers temporarily captured at relay nodes that have lost their delivery path to the root. The replay cache (next slide "*Temporal pub/sub*") was designed to close exactly this gap.
3. **Axon-role count grows with churn but plateaus.** From 537 axons at 0 % churn to ~2,200 at 30 %, leveling off. The tree absorbs growth into deeper structure rather than unbounded fan-out.

Through 20 % cumulative churn, Axona holds delivery above 86 % with no replication, no gossip, and no parent tracking. *Above 25 % churn the protocol enters a recovery-paced regime where steady-state delivery is the equilibrium of recruitment vs. loss.*

Honesty: locality vs. latency

Axona's priorities are speed and robustness — fast lookup, reliable delivery. Geographic location is *not a goal*; it is one input.

What the simulator does. Every protocol's hop latency is computed from physical (haversine) distance between simulated nodes — a faithful Internet model. This is what the real Internet looks like to all four protocols.

What the protocol does. A node knows *only its own* S2 cell — embedded once, at creation, into the top of its 64-bit ID:

```
nodeId = S2 cell prefix (8 bits) ‖ H(publicKey)
```

The protocol cannot ask another node "*where are you?*" — there is no such message. The S2 prefix creates **initial regional clustering at bootstrap** because nearby nodes are XOR-close, so a node's K-bucket and synaptome are seeded with mostly-local peers. After that, locality is a derived property of the routes that LTP and vitality reinforce.

Latency is used in routing, indirectly: AP scoring includes a `½^(latency_ms/100)` penalty, so observed round-trip time (**RTT**) influences hop selection. Fast peers win at every distance — that is the speed-first priority in action.

Lineage note. *Putting locality in the routing layer itself* is the canonical contribution of the parallel **Tapestry** (Zhao et al., JSAC 2004) and **Pastry** (Rowstron & Druschel, Middleware 2001) lineages — both store the closest match by RTT in each routing-table slot, and Pastry's locality-preserving join carries the property to new nodes without exhaustive search. Axona inherits the instinct from both and pushes it further: **structural** (S2 prefix at bootstrap, three-tier role decomposition via vitality) plus **dynamic** (latency penalty in AP scoring, evolving via LTP). Both axes are descended from the Pastry / Tapestry choice; the *learning* is what we add.

The geoBits = 0 ablation

When we strip the S2 prefix from **NH-1** and re-run, regional latency collapses:

Test	NH-1 · geoBits = 8 (hops / ms)	NH-1 · geoBits = 0 (hops / ms)	Δ ms
Global	5.09 / 263 ms	5.94 / 453 ms	+72 %
500 km	3.31 / 96 ms	**6.01 / 444 ms**	**+363 %**
2000 km	3.93 / 123 ms	6.01 / 437 ms	+255 %
5000 km	4.54 / 169 ms[1]	5.96 / 433 ms	+156 %

All cells are NH-1 measurements at 25 K nodes, web-limited (cap = 100), hw = 0. The 4-way comparison covering all protocols at geoBits = 0 is on the next slide.

[1] The geoBits = 8 baseline at 5000 km is from the headline 4-way comparison run (which measured all five distance bands). The geoBits = 0 sweep run measured 5000 km natively; the geoBits = 8 run in the same sweep skipped 5000 km for time. Numbers are from comparable 25 K, hw = 0 conditions but distinct CSVs.

The latency penalty in AP routing is not strong enough, on its own, to discover locality from scratch within standard warmup (~5000 lookups). The S2 prefix is the **bootstrap shortcut**: it does not *teach* locality, it *seeds* the network with locality so reinforcement can sharpen routing within it.

Geography is an initial-condition shortcut for the latency optimization, not a goal in itself. Future work: an embedded RTT-coordinate system (Vivaldi-style) could replace the S2 seed with a learned one — see the comparison slide later in this deck.

geoBits = 0 — isolating the learning advantage

If we strip the S2 prefix from *every* protocol simultaneously — and use **canonical init** so each protocol starts from the identical K-closest-XOR routing table — the comparative picture isolates exactly the routing/learning algorithm.

25 K nodes, web-limited (cap = 100), omniscient init, **geoBits = 0**, **canonical init**.

Protocol	geoBits = 8 · Global ms (reference)	geoBits = 0 · Global ms	geoBits = 0 · 500 km	geoBits = 0 · 2000 km	Notes
Kademlia	508	**506**	511	504	No locality either way — geoBits doesn't matter to pure XOR.
G-DHT	284	**780**	765	781	Geo-prefix carries G-DHT entirely; without it, a `noProgressLimit = 3` overhead leaves it slower than K-DHT.
NX-17	241	**376**	**341**	**355**	Loses locality but still beats Kademlia by **26 %** at geoBits = 0 — pure learning advantage.
NH-1	269	467	418	439	Beats Kademlia by **8 %** at geoBits = 0. Same learning chassis as NX-17, simpler — pays a measurable consolidation cost.

The first column repeats each protocol's geoBits = 8 baseline so the *learning advantage* and *locality contribution* can be read off the same row. Headline finding is the third column (geoBits = 0 global).

Three findings:

1. **NX-17 and NH-1 still beat Kademlia under identical bootstrap and no locality.** This is the cleanest possible "learning helps" claim — every confound (bootstrap strategy, geographic prefix) controlled. NX-17 wins by 26 %, NH-1 by 8 %.
2. **G-DHT vs K-DHT exposes a non-bootstrap difference.** Canonical init equalizes bootstrap; the residual 780-vs-506 gap comes from G-DHT's lookup-tuning choice (`noProgressLimit = 3`), tuned for geographic-routing escapes that don't exist at geoBits = 0. A real architectural finding the methodology surfaced.
3. **NH-1 vs NX-17 narrows but doesn't close.** The 80/20 supplement helped NX-17 by ~7 ms at geoBits=8 (native vs canonical). At geoBits=0 the 376-vs-467 gap is roughly 24 % — it's NX-17's specialized rules, not its bootstrap, doing the work.

The headline gap is real and measurable. *NX-17 at 376 ms vs Kademlia at 506 ms, identical starting state, no locality, no geography — that is what the routing/learning algorithm contributes. Removing that distance from the deck would be hiding the actual claim.*

Init mode × geoBits — the full 2 × 2

Reference table — the full bootstrap × locality cross-product. The earlier geoBits=0 ablation slide carries the headline finding; this slide is for readers who want to verify the result holds across all four cells.

Global lookup latency in milliseconds. **Canonical** = every protocol starts from K-closest-XOR (identical state). **Native** = each protocol's own bootstrap strategy (Kademlia: pure XOR; G-DHT: 50/50 stratified+random; NX-17: 80/20; NH-1: pure XOR).

25 K nodes, web-limited (cap = 100), omniscient init, hw = 0.

	Kademlia	G-DHT	NX-17	NH-1
geoBits = 8 · native	488	289	**234**	254
geoBits = 8 · canonical	508	284	241	269
geoBits = 0 · native	493	769	377	461
geoBits = 0 · canonical	506	780	**376**	467

Three things this 2 × 2 makes visible:

- **Bootstrap matters less than people often assume.** Native vs canonical at geoBits = 8 swings each protocol by under 8 %. The architectural choices are real but small relative to other effects.
- **The geographic prefix matters a lot — for everyone.** Geo = 8 to geo = 0 doubles K-DHT-equivalent latency. The S2 prefix is doing real work in every column. Removing it forces every protocol to route under genuinely random IDs.
- **Learning matters most under stress.** At geoBits = 0 / canonical (the hardest condition for routing), NX-17 is 26 % faster than Kademlia and NH-1 is 8 % faster. Under easier conditions (geoBits = 8) the gap widens further. **The advantage scales with how much there is to learn.**

Canonical-init isolates the routing/learning algorithm. Native-init measures the protocol as deployed. Both are real; both belong in the record. The deck headlines canonical because it's the cleaner architectural claim.

N-1 → Axona (NH-1) — three phases of research

Phase	Generation	Contribution
Exploration	N-1	First neuromorphic DHT — Hebbian synapse weighting, action-potential routing, two-hop lookahead
	N-15W	Renewal-Based Highway Protection — last protocol of the original lineage before the restart
Focused iteration	NX-1	Clean, configurable restart of the architecture from a fresh base
	NX-4	**Watershed**: iterative fallback — every protocol below fails under stress, every protocol at or above succeeds
	NX-6	Churn-resilience baseline: dead-synapse eviction, temperature reheat, adaptive decay
	NX-10	Routing-topology forwarding tree — the first stable axonal-style pub/sub
	NX-11	Diversified bootstrap — the 80 / 20 stratified + random allocation that NX-17 inherits
	NX-15	`AxonManager` generic membership protocol with K-closest replication
	NX-17	Publisher-prefix topic IDs + pure routed axonal tree (no replication, no gossip)
Consolidation	**NH-1**	Vitality-driven unified admission gate — consolidates 18 specialized rules into 12; ~270 lines vs NX-17's ~2300

NX-16: a cautionary dead end. *NX-16 attempted to decouple* `findKClosest` *'s target cell from the node-ID prefix by masking the top 8 bits in the distance metric. Routing collapsed: publisher and subscribers found different "closest" nodes for the same topic; delivery dropped to ~40 % even at zero churn. The lesson:* ***the distance metric used to select candidates must match the gradient used to expand them.*** *Archived, not deleted — design failures earn their place in the record.*

NX-17 rules and their Axona (NH-1) disposition

Reference table — the full audit trail of which NX-17 rules survived consolidation and which were dropped. Skip on a first read; consult when you want to know what specifically changed.

NX-17 rule	What it does	NH-1 status
Stratified bootstrap	Cold-start coverage across XOR strata	**Kept** (STRUCTURE)
Diversified bootstrap (80/20)	20 % random peers for global reach	**Kept** (merged into bootstrap)
Two-tier synaptome (50 + highway)	Separate "permanent" + "candidate" tiers	**Replaced** by per-node `_maxSynaptome`
Stratified eviction	Evict from over-represented stratum	**Replaced** by vitality eviction
Stratum floors	Guarantee min peers per stratum	**Removed** — vitality + diversity penalty handles it
Synaptome floor	Refuse eviction below N peers	**Removed** — vitality preserves locked entries
AP routing	Learned-weight greedy walk	**Kept** (NAVIGATE)
Two-hop lookahead	Probe second-hop alternatives	**Kept** (NAVIGATE)
Epsilon-greedy first hop	Random first hop with prob ε	**Kept** (EXPLORE)
Iterative fallback	k-closest fallback when AP fails	**Kept** (NAVIGATE)
LTP	Reinforce successful paths	**Kept** (LEARN)
Hop caching + lateral spread	Cache destination on intermediates	**Kept** (LEARN)
Triadic closure	Learn long edges from transit	**Kept** (LEARN)
Incoming promotion	Promote popular incoming peers	**Kept** (LEARN)
Adaptive decay	Per-node decay rate based on usage	**Default off** — flat γ = 0.995 wins
Simulated annealing	Replace lowest-vitality with 2-hop sample	**Kept** (EXPLORE)
Churn recovery (dead-syn eviction)	Drop dead peers on discovery	**Kept** (FORGET)
Temperature reheat on death	Spike T after dead-peer discovery	**Kept** (EXPLORE)
Diversity budget penalty	Penalise over-represented stratum groups	**Removed** v0.65.06 (harmful)
weightScale parameter	Bias AP scoring toward LTP weight	**Removed** v0.66.10 (no measurable effect)

~18 rules + 5 retired parameters → 12 active rules in NH-1. Behavioral surface is the same; admission logic is unified.

Axona (NH-1) vs NX-17 — and why

Test	NX-17	NH-1	Δ (NH-1 vs NX-17)
Global	4.40 hops / 242 ms	5.15 hops / 263 ms	+17 % hops, +9 % ms
500 km	2.75 / 80 ms	3.28 / 95 ms	+19 % hops, +18 % ms
2000 km	3.27 / 105 ms	4.01 / 126 ms	+23 % hops, +20 % ms
5000 km	3.74 / 143 ms	4.54 / 169 ms	+21 % hops, +18 % ms
pubsubm delivered	100 %	100 %	tie
pubsubm + 5 % churn (recovered)	100 %	100 %	tie
dead-children / orphans	0 / 0	0 / 0	tie

The residual gap reflects two things:

1. NX-17's specialized eviction policies are tuned for the cap=100 case. Stratified eviction with stratum and synaptome floors *guarantees* a coverage profile that NH-1's vitality eviction approximates but doesn't pin. Under tight caps, that difference is measurable.

2. NH-1 trades peak performance for simplicity. ~270 lines vs NX-17's ~2300; 12 parameters vs ~44. The point is not that NH-1 is *faster* — it's that NH-1 is *almost as fast* with one unified admission gate that's easier to reason about and easier to extend.

*Highway% recovers the gap and more — at hw=15 % NH-1 is **223 ms global**, beating NX-17's all-browser 242 ms. The capped gap is the cost of consolidation. We accept it.*

What is Kademlia?

Maymounkov & Mazières (NYU — IPTPS 2002, *Kademlia: A Peer-to-peer Information System Based on the XOR Metric*) is **the foundational paper Axona is built on**. Every Kademlia idea — the XOR metric, K-buckets, α-parallel lookup, the four-RPC protocol — survives literally inside Axona's NH-1 implementation. Without Kademlia, there is no Axona.

The XOR metric. Distance between two 160-bit IDs is `d(x, y) = x ⊕ y` — the bitwise XOR interpreted as an integer. **Symmetric** (`d(x,y) = d(y,x)`) and obeys the triangle inequality. Symmetry is what makes Kademlia self-organizing: every message a node receives — request *or* reply — conveys useful contact information that updates the recipient's routing table. Asymmetric metrics (Chord's clockwise distance) force one-way table maintenance and rigid finger-table positioning.

K-buckets. Each node maintains up to **K=20 peers** per XOR-distance interval `[2^i, 2^(i+1))`. Lists are sorted by last-seen — least-recently-seen at the head. New contacts are inserted at the tail; **live old contacts are never evicted**, only stale ones. This LRU-with-old-bias policy exploits a key empirical observation (Saroiu et al.): *the longer a peer has been up, the more likely it stays up another hour.* Old contacts are the reliable ones.

α-parallel lookup. To find the K nodes closest to a target ID, a node issues **α=3 parallel asynchronous `FIND_NODE` RPCs** to its α closest known peers. Each response narrows the candidate set; the node iterates until the K closest seen are all reachable. Parallelism trades a constant-factor bandwidth increase for **delay-free fault tolerance** — if one query times out, two others are already in flight.

Four RPCs: `PING`, `STORE`, `FIND_NODE`, `FIND_VALUE`. The entire protocol surface fits in this set.

Production deployments. BitTorrent Mainline DHT (millions of simultaneous nodes since 2005), Ethereum devp2p (peer discovery for blockchain consensus), IPFS / libp2p (content routing), Tor v3 hidden services (onion-service descriptor lookup). **The most-deployed DHT family in production.**

For this project: *Kademlia is Axona's literal substrate. Every learning rule, every vitality calculation, every axonal tree sits on top of Kademlia routing. When AP scoring fails (NX-4 iterative fallback), the protocol falls back to unmodified Kademlia. The relationship is not analogical — it is **structural inclusion**.*

Kademlia vs Axona

The most important comparison in this section. **Axona does not replace Kademlia — it adds learning on top of it.**

Aspect	Kademlia	Axona
Distance metric	XOR, symmetric, triangle-inequality	**Same XOR metric** — inherited unchanged
Routing-table structure	K-buckets, K=20 per stratum	**K-buckets become the synaptome** — same per-stratum layout, K capped at 50 total entries
Eviction policy	LRU within bucket; live old peers never evicted	**Replaced by vitality** (`weight × recency`) — generalizes the "old-is-better" heuristic to traffic-driven evidence
Lookup parallelism	α-parallel async `FIND_NODE` (α=3)	**Same α-parallel pattern** retained for iterative fallback; AP scoring narrows the parallel set
Routing decision	Pick α closest by XOR	**Pick best by AP score** = `progress × weight × ½^(latency/100)` — XOR is one of three factors, not the only one
Self-organization	Every incoming/outgoing message updates a K-bucket	**Same** — plus LTP reinforcement on successful paths
Pub/sub	Out of scope	First-class — axonal trees on top of Kademlia routing
Locality	None — IDs are random hashes	S2 prefix at bootstrap + AP latency penalty + LTP reinforcement

Where they overlap. Everything below the synaptome layer is Kademlia. XOR distance, K-buckets, α-parallel lookup, the four-RPC protocol surface, the symmetric-metric self-organization, the "trust old peers" heuristic — all retained literally. NX-4 iterative fallback is the path the protocol takes when learning provides no signal: pure Kademlia.

Where Axona continues. Three additions, in order of impact: **(1)** synapses carry weights reinforced by LTP (Hebbian learning replaces XOR-only ranking); **(2)** S2 prefix in IDs gives bootstrap locality (Kademlia is locality-blind); **(3)** axonal trees provide a built-in pub/sub primitive (Kademlia has no broadcast).

The relationship is structural, not analogical. *Axona's value proposition is "everything Kademlia does, plus learning." The 3δ-floor result and the Slice World recovery are improvements on top of a working Kademlia substrate — not replacements for it. When you read the NH-1 source code, you read Kademlia idioms throughout.*

What is Pastry / SCRIBE?

Rowstron & Druschel (Microsoft Research + Rice — Middleware 2001, *Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems*) is the **direct sibling of Tapestry** — same era, same family (prefix-based + locality-aware), substrate for **PAST** (persistent storage) and **SCRIBE** (pub/sub). Together Pastry and Tapestry are the **two foundational lineages** for any locality-aware adaptive DHT.

The defining contribution: three-tier routing state. Each Pastry node maintains:

Structure	Purpose	Size at b=4
Routing table R	Prefix-matched routing across `⌈log_(2^b) N⌉` rows × `2^b − 1` entries	~75 entries at 1 M nodes
Leaf set L	Numerically-closest peers; used for the **final routing hop**	16–32 entries
Neighborhood set M	RTT-closest peers; **not routed through** but kept current to maintain locality	16–32 entries

Locality-preserving join. When a new node X joins, it asks a nearby existing node A to route a "join" message with key=X. Each node along the path sends X its tables; **X takes row n of its routing table from the n-th node along the route**. Triangulation-inequality argument preserves locality without exhaustive search. A second-stage refinement queries each entry's own tables for closer alternatives.

SCRIBE pub/sub (Castro, Druschel, Kermarrec, Rowstron — JSAC 2002, layered on Pastry): `topicId = hash(name)` → subscribe routed to the rendezvous point; **every node along the path records subscriber state**; publisher sends to rendezvous; multicast tree formed by the **reverse paths** of all subscribes. **This is structurally identical to Axona's axonal-tree mechanism** — same recipe, our terminology.

Headline metrics. ~5 hops at 100 K nodes (= `⌈log_16 100K⌉`); RDP ~1.3–1.4× the complete-routing-table optimum; locates closest of k=5 replicas 76 % (92 % top-2) of the time; 57 RPCs to repair tables per failed node; ~3000 msg/s per Java node, unoptimized.

For this project: *Pastry contributed the **three-tier routing state** that Axona's vitality-scored synaptome consolidates, and **SCRIBE** is the structural ancestor of axonal-tree pub/sub. Pastry is the structural sibling of Tapestry's architectural contribution.*

Pastry vs Axona

The cleanest one-paragraph framing: **Axona's synaptome is Pastry's R/L/M collapsed into a single tier scored by `weight × recency`. The roles persist; the bookkeeping is unified.**

Aspect	Pastry	Axona
Routing state	**Three explicit tables** (R, L, M)	**One vitality-scored synaptome** that places each peer into its effective tier
Leaf set role	Numerically-closest 16–32 peers, terminal-hop guarantee	Synaptome's **high-vitality core** — well-trafficked nearby peers, dominate terminal-hop routing
Routing table role	Prefix-matched coverage, ~75 entries	**Mid-vitality** entries distributed across XOR strata
Neighborhood set role	RTT-close peers kept for locality maintenance	**Low-vitality entries + 2-hop annealing pool** — candidate replacements, not primary routes
Locality	Static after locality-preserving join; lazy repair	Continuously evolved via LTP, triadic closure, hop caching, lateral spread
Pub/sub	SCRIBE — routed subscribe + reverse-path forwarding tree, layered on top	**Axonal trees** — same mechanism, built into the protocol; tree adapts via re-subscribe
Bootstrap	Locality-preserving join (take row n from n-th hop)	Sponsor-chain join — **directly inherited from Pastry**, generalized to XOR strata

Where they overlap (deeply). Three concrete inheritances: **(1)** the three-tier routing state, even if Axona collapses it; **(2)** locality-preserving join — Pastry formalized it, we still use it; **(3)** SCRIBE → axonal trees — *the same mechanism*, named differently.

Where Axona continues. Pastry's locality is *static after bootstrap*; ours *evolves* via LTP. SCRIBE trees are *fixed by routing topology*; our axon trees *re-shape* via re-subscribe and incoming promotion. Pastry's leaf set is fixed-size and ID-proximal; our high-vitality core is *traffic-earned*.

Pastry is the structural ancestor; Tapestry is the architectural ancestor. *Together they give us the two foundational lineages, and Axona sits at the synthesis point: Pastry's three-tier structure (consolidated into vitality), Tapestry's locality-aware routing (made dynamic via AP scoring), Pastry's SCRIBE (made adaptive via re-subscribe), Tapestry's soft-state repair (event-triggered + Patchwork). Axona = Berkeley + Cambridge + learning.*

What is Tapestry / DOLR?

Zhao, Huang, Stribling, Rhea, Joseph, Kubiatowicz (UC Berkeley + MIT — JSAC 2004, *Tapestry: A Resilient Global-Scale Overlay for Service Deployment*) is **the foundational paper on putting locality, soft-state, and resilience in the routing layer itself**. It was the substrate for OceanStore, Bayeux multicast, Mnemosyne steganographic storage, and Spamwatch — the closest-shaped historical ancestor of the entire NH-1 architecture.

Two core ideas:

- **Prefix routing with locality optimization.** Like Pastry, Tapestry routes by progressively matching one more digit of the destination ID per hop (base β=16, $\log_{16} N$ hops). The Tapestry-specific contribution: at each routing-table slot, the entry stored is the **closest node by RTT** that matches the prefix at that level. The routing table is built at node insertion via iterative nearest-neighbor search.
- **DOLR — Decentralized Object Location and Routing.** A different API from "DHT": `PublishObject`, `RouteToObject`, `RouteToNode`. When a server stores object O at GUID, it routes a *publish* message toward O's deterministic root; **every node along the publish path stores `<GUID, server>` as a soft-state location pointer.** Queries route toward the root and intersect the publish path early. The "tree" of replica locations *emerges from the union of publish paths*, not from explicit construction.

Headline metric — RDP (Relative Delay Penalty) = `Tapestry route latency ÷ direct IP latency`. Median ≈ 1 in the wide area; 90th percentile ~2–3. With `(k backups, l nearest, m hops)` location-pointer optimization, 90th-percentile RDP drops below 4 even at short distances. **~100 % routing success under massive failures (20 % kill, 50 % join) and continuous churn (4-min mean lifetime).**

For this project: *Tapestry is not a parallel mechanism to compare against — it is the **historical precedent** for three of Axona's four design pillars (locality, soft-state, resilience). Where it stops, our learning continues; where it ends with Bayeux-on-top, our pub/sub is built-in.*

Tapestry vs Axona

The closest-shaped ancestor — and the clearest "what we inherit, what we add" comparison.

Aspect	Tapestry / DOLR	Axona
Routing primitive	Prefix matching, base β=16	Kademlia XOR (Kademlia base-2 strata)
Locality in routing table	**Closest node by RTT** stored per prefix slot, built at insertion	**Latency-penalized AP score** evaluated dynamically per lookup; S2 prefix in IDs structurally seeds locality
When locality is set	Once, at node insertion (RTT-optimized routing table)	Continuously — every successful lookup reinforces locality via LTP
Replica location	Soft-state pointers along publish paths; query intersects	Axonal trees rooted at deterministic topic ID; subscribers attach during routed subscribe
Surrogate routing *(canonical term — Tapestry's name)*	Built-in: when deterministic root is dead, route to closest live ID	NX-4 *iterative fallback* — same mechanism, different name
Resilience	Soft-state mesh repair: event-triggered + Patchwork background probes	Temperature reheat on dead-peer discovery; vitality eviction; iterative fallback
Pub/sub	Layered on top — Bayeux multicast	Built-in — axonal delivery trees
Deployment	Server-class Java on PlanetLab (~100 machines)	Browser-class WebRTC, simulator targeting 50 K nodes

Where they overlap (deeply). *Soft-state location pointers along the publish path* is structurally identical to *axonal subscribe-attach along the routed path*. *Surrogate routing* is exactly NX-4 iterative fallback. *Mesh repair* (event + Patchwork) is the same family as our reheat + anneal. *Multiple backup pointers per slot* is what our weighted synaptome candidates do at the slot level. The deeper overlaps are not coincidence — they are *Axona inheriting Tapestry's architectural choices and pushing them further with learning*.

Where Axona continues. Tapestry's locality is *static after bootstrap*; ours is *continuously evolved* via LTP, triadic closure, hop caching, lateral spread. Tapestry's pub/sub was layered (Bayeux); ours is built-in. Tapestry's deployment was server-class; ours targets browser-WebRTC.

The lineage is direct. *Tapestry put locality and resilience in the routing layer; NH-1 makes those properties learn from traffic. Citing Tapestry prominently doesn't dilute our claim — it positions our work in continuity with two decades of validated foundational research. The Berkeley OceanStore family is the closest-shaped ancestor; we are an evolutionary step on that line, not a clean-room reinvention.*

What is adaptive stabilization?

Ghinita & Teo (National University of Singapore — IPDPS 2006, *An Adaptive Stabilization Framework for Distributed Hash Tables*) addresses one of the longest-running problems in DHT design: **how often should a peer check that its routing table is still correct?**

Most DHTs (Chord, Pastry, CAN) run **periodic stabilization** — a fixed-interval timer that pings neighbors and refreshes pointers. Their core claim: *a fixed rate is wrong almost everywhere*. Too low → lookup failure spikes during churn bursts (their data: 23.7 % failure at 5 / sec churn). Too high → 400 %+ communication overhead even during quiet periods.

Mechanism: each peer estimates local conditions, models pointer staleness probabilistically, and triggers checks only when warranted. Per peer:

- Estimate **node failure rate μ** and **node join rate λ** locally (rolling window of observations)
- Estimate **network size N** from successor-list density
- Compute `P_dead(p) = 1 − exp(−μ × Δt)` (probability pointer p's target died) and `P_inacc(p)` (probability a new joiner now sits between p and its ideal target)
- **Split stabilization into two channels with very different costs:**
  - **Liveness check** — O(1) ping, fires when `P_dead × P_fwd > threshold`
  - **Accuracy check** — O(log N) lookup, fires when `P_inacc > threshold`
- Operate against a single tunable knob: **target lookup failure rate** `Pf`

Headline result. On Chord with variable churn (peak-hour pattern), Adaptive Stabilization (AS) hits **2.2 % peak failure at 172 % overhead** — vs Periodic Stabilization's **7.5 % failure at 420 % overhead**. **3.4× lower failure, 2.4× lower overhead** simultaneously, by self-tuning rather than hand-picking a rate.

For this project: *this is the closest match in mechanism family of any DHT comparison we make. The instinct — "the protocol should adapt itself based on local observation, not run on a fixed schedule" — is exactly the Neuromorphic instinct. The difference is what they tune (maintenance rate) vs what we tune (synaptome composition).*

Adaptive stabilization vs Axona

Both reject fixed-schedule maintenance. Different observables, different actions.

Aspect	Ghinita & Teo (Adaptive Stabilization)	Axona
What's adapted	Stabilization rate per routing pointer	Synaptome composition (which peers are kept)
Trigger	`P_dead` or `P_inacc` exceeds threshold	Dead-peer discovery (reheat); per-lookup probabilistic anneal
Local observable	Node failure rate **μ**, join rate **λ** estimated from routing table	Per-synapse `weight × recency`; observed dead peers
Statistical model	Closed-form: `P_dead = 1 − exp(−μΔt)`, `P_inacc` from new-joiner density	None — vitality is a heuristic correlate, not a probabilistic model
QoS knob	**Yes — single dial `Pf` (target lookup failure rate)**	**No — parameters are tuned by sweep, not by a target**
Liveness vs accuracy	**Explicit decoupled channels** with different cost / threshold	**Implicit, mixed** — both fire as side-effects of routing
What survives	Routing pointers (back to canonical-XOR ideal)	Routing pointers (toward traffic-shaped optimum)

Where they overlap. Decentralized self-monitoring + threshold-triggered protocol response — the same control-loop architecture as our **temperature reheat** (NX-6), Makris's **DFE migration**, and the proposed **load-aware AP scoring**. *Four independent works converging on the same architectural pattern* is meaningful evidence that this is a recognizable design class, not a one-off choice.

Where they differ. Ghinita-Teo is purely *statistical* — derive analytical formulas, estimate parameters, threshold. Axona is *biological* — synaptic weights, vitality, decay. Both work; both have limits. **The synthesis would be Axona with Ghinita-style statistical estimation of churn driving adaptive anneal/reheat parameters.**

The headline trade. *Their stabilization brings a Chord routing table back to its canonical ideal under churn. Our learning evolves the routing table away from the canonical ideal toward a traffic-shaped one.* ***Combining the two — Ghinita's adaptive cadence with Axona's adaptive content — is the metaplastic Axona we keep gesturing at.***

What is Coral DSHT?

Coral (Freedman, Freudenthal, Mazières — NSDI 2004) is a **distributed sloppy hash table** that powered Coral CDN — one of the earliest production decentralized content-distribution networks. Coral coined the term *DSHT* to distinguish itself from strict DHTs.

Mechanism — hierarchical RTT clusters. Each node measures round-trip time to every other node it encounters and joins **multiple nested clusters** at increasing RTT thresholds (e.g. < 20 ms, < 60 ms, global). Each cluster runs its own internal DHT.

Lookup is local-first. When a node queries a key, it first searches the tightest cluster it belongs to. If no result is found there, it expands outward to the next cluster, and so on. Most queries terminate in the local cluster — cross-cluster fan-out happens only when necessary.

"Sloppy" semantics. The strict DHT promise — *one canonical XOR-closest node per key* — is deliberately relaxed. A key may be stored at *any* node in the lookup cluster, and queries return the *first* hit, not the canonical one. This is what makes Coral a CDN: the same content is replicated at many local nodes, and clients pull from the nearest available.

Production deployments: Coral CDN (2004 – ~2015) — handled millions of cache requests per day at peak, served as a free CDN for academic and small-publisher sites.

For this project: *Coral's hierarchical-cluster idea is parallel to Axona's weighted-synaptome approach. Both achieve latency-aware routing; they make very different structural commitments to get there.*

Coral vs Axona

Both adapt to network latency. They make opposite structural choices.

Aspect	Coral DSHT	Axona
Structure	Multiple nested DHTs, one per RTT cluster	A single flat synaptome with weighted edges
Locality discovery	Active RTT measurement at join + cluster membership	Passive — observed traffic reinforces useful edges; S2 prefix seeds initial regional bias
Storage semantics	*Sloppy* — multiple replicas per key, return-first-found	*Strict* — one canonical XOR-closest node per key
Lookup	Local cluster first, escalate outward only if needed	Single greedy AP walk over weighted synaptome
Adaptation	Cluster boundaries are static thresholds; nodes move between clusters	Continuous: weights update on every successful path
Designed for	Read-heavy content distribution (CDN) — many readers, mostly-static content	Routing + pub/sub — dynamic membership, real-time delivery
Pub/sub support	Out of scope	First-class via axonal trees

Where they overlap: both reject the "one fixed routing structure for all distances" approach. Both treat geographic locality as a property to *exploit*, not just record.

Where they differ: Coral *enforces* locality through the hierarchical cluster structure — every node *knows* what's near. Axona *learns* locality through usage — every node *discovers* what's near via routed traffic. Coral relaxes the DHT promise to be local-first; Axona preserves the strict DHT promise and learns shortcuts on top.

Coral's design choice was: "give up the canonical mapping to win locality." Axona's design choice was: "keep the canonical mapping; make locality emerge from learning." Different engineering trade-offs against the same core problem.

What is Vivaldi?

Vivaldi (Dabek, Cox, Kaashoek, Morris — SIGCOMM 2004) is a **decentralized network coordinate system**. Each node continuously adjusts a synthetic position vector — typically in 3-dimensional Euclidean space plus a small "height" component — so that the *Euclidean distance* between any two nodes' coordinates approximates the *measured round-trip time* between them.

Mechanism. Each node starts at a random position. Whenever it exchanges traffic with a peer, it observes the actual RTT, then applies a small spring-force adjustment to its own coordinate — pulling toward or pushing away from that peer to better match observation. Confidence in each measurement weights the magnitude of the move. Over many such observations, the system converges to a stable configuration where pair-wise Euclidean distances ≈ pair-wise RTTs.

What it accomplishes. Any node can *predict* RTT to any peer it has never directly measured, purely from coordinates. Routing layers (or applications) can then choose "nearby" peers without an active probe round — Vivaldi turns latency into a queryable global property of the network. The system is fully decentralized: no coordinator, no infrastructure, no synchronization.

Production deployments: Coral CDN, Azureus / Vuze (BitTorrent), some content-addressable overlays. The most influential predecessor for *latency-aware* peer-to-peer design.

For this project: *a Vivaldi-style coordinate system is a candidate for replacing Axona's structural S2 prefix with a learned locality primitive. Future benchmarks may include a Vivaldi-style protocol for completeness — both as a comparison reference and as a forward-looking direction.*

Vivaldi vs Axona

Both are adaptive routing systems. They optimize different things.

Aspect	Vivaldi	Axona
Mechanism	Synthetic coordinates in N-D Euclidean space	Hebbian reinforcement on routing edges
What's learned	A position vector per node that predicts RTT to any peer	A weight per synapse, reinforced by traffic on successful paths
Output	RTT prediction (any pair)	A ranked list of next-hops (per lookup)
Locality discovery	Emergent from RTT measurements	Imposed via S2 cell ID prefix
Convergence guarantee	Yes — coordinate descent on RTT residuals	Empirical — depends on traffic mixing
Layered separation	Coordinate system is generic; any routing layer can use it	Routing and learning are integrated in a single protocol
Pub/sub support	Out of scope — Vivaldi is a primitive	First-class — axonal tree built on routed synaptome

Where they overlap: both treat the network as something to *measure and adapt to*, not a fixed topology to traverse.

Where they differ: Vivaldi predicts RTT, then any routing layer uses those predictions. Axona learns *paths* via reinforcement on the routing layer itself — locality is built into the IDs, learning sharpens the path within that locality.

What is route-diversity replication?

Castro, Druschel, Ganesh, Rowstron, Wallach (Microsoft Research / Rice — OSDI 2002, *Secure Routing for Structured Peer-to-Peer Overlay Networks*) is the foundational paper on **Byzantine fault tolerance** in DHTs. They prove that three jointly-necessary mechanisms make a DHT robust against malicious nodes: **constrained routing tables**, **secure node ID assignment**, and **redundant routing**. The third is what later work elaborates.

Harvesf & Blough (Georgia Tech — IEEE P2P 2007, *The Design and Evaluation of Techniques for Route Diversity in Distributed Hash Tables*) makes the redundant-routing piece concrete with a clean theorem:

To produce d disjoint routes from any source to a key k in a prefix-matching DHT with base B, replicate k at (n+1) × B^m locations, with `m = ⌊(d−1)/(B−1)⌋` and `n = (d−1) mod (B−1)`.

The replicas are placed by **varying the length of common prefix** among replica IDs — so that any two routes from a query node to the replica set diverge at the first hop and never share an intermediate. Two complementary techniques:

- **MaxDisjoint replica placement** — the placement formula above. Multiple disjoint paths to the *content*.
- **Neighbor Set Routing (NBR)** — issue the lookup through the source's *neighbors'* tables, not just your own. Multiple disjoint paths from the *source*.

Empirical headline (1024-node Pastry, 8 replicas, 100 K lookups): with **half the network compromised**, MaxDisjoint + NBR routes **90 % of lookups successfully** — vs ~66 % for replica placement alone, and ~50 % for random placement.

For this project: *route-diversity replication is the canonical answer to the Byzantine-resistance gap our red-team analysis flags. The mechanism is well-studied; what we'd add is specializing it for axonal pub/sub topic roots and S2-cell eclipse defense.*

Route-diversity DHTs vs Axona

Both families care about routing robustness. They commit to different mechanisms.

Aspect	Castro / Harvesf-Blough	Axona
Threat model	Byzantine — malicious nodes that lie about routing	Crash-failure — honest peers that disappear
Mechanism	Replicate the *target* at d disjoint placements; query in parallel	Maintain a weighted *synaptome* with overlapping candidates per logical destination
What's redundant	Multiple disjoint *paths to the same key*	Multiple weighted *next-hop options* per lookup
Activation	Always — every lookup queries replicas in parallel (or in batches)	Reactive — iterative fallback only fires when greedy AP routing dead-ends
Cost	×d storage, parallel network load per lookup	One synaptome; lookup load unchanged
Pub/sub support	Out of scope	First-class via axonal trees — but currently single-replica root

Where they overlap. Both reject "one route is enough." Both treat the routing fabric as something whose redundancy can be *engineered for* rather than hoped for.

Where they differ. Castro / Harvesf-Blough is **storage-side redundancy** (the same key lives in d places, queries are parallel). Axona is **synaptome-side redundancy** (one key lives in one place, the routing table holds many candidates per direction). The two are complementary, not competing — a real production system would likely use both.

The headline trade. *Their work delivers strong Byzantine resilience (90 % success at 50 % malicious) at the cost of d× storage and parallel query load. Axona delivers strong crash-failure resilience (100 % delivery under 5 % churn) at the cost of zero extra storage. Combining the two — MaxDisjoint replication of Axona axon-tree roots — is the obvious next step for a Byzantine-tolerant pub/sub.*

What is hotspot-aware placement?

Makris, Tserpes, Anagnostopoulos (Harokopio Athens — IEEE BIGDATA 2017, *A novel object placement protocol for minimizing the average response time of get operations in distributed key-value stores*) addresses a problem that consistent-hashing DHTs systematically ignore: **request rates are not uniform even when keys are.**

Real workloads follow **Zipf's law** — a small set of "hot" keys takes most of the traffic. Their measurement on a 24-node Redis cluster: even with keys evenly distributed, the hottest node received **222 K requests** while others received ~1–10 K each. Response time on the hotspot was 5× the cluster median. **Consistent hashing solves the wrong half of the problem.**

Mechanism: Directory For Exceptions (DFE). A hybrid placement that keeps consistent hashing as the *default* and adds a small distributed override:

1. Each node monitors its own **average response time (RT)** and **request count (NR)**
2. When `max(NR) > permissible threshold T`, the node identifies its hottest keys
3. Picks the **least-loaded peer** in the cluster (offline FFD bin-packing chooses placements)
4. **Migrates** the hot key there
5. Installs a **DFE entry** — other nodes consult DFE before falling back to the hash function

Fully decentralized: each node operates independently, no central coordinator.

Headline result. Hotspot N3's response time **dropped 86 %**; load uniformized across the cluster (~10 K – 25 K requests per node, down from 222 K on N3). The other nodes saw response time rise ~3.9× — but stayed well below the threshold.

For this project: *the hot-key migration pattern maps directly onto Axona's pub/sub gateway concentration failure mode. A topic with millions of subscribers is the same engineering problem as Redis's hot key — and the DFE mechanism is the family of answers that fits.*

Hotspot-aware vs Axona

Both care about routing performance under *non-uniform* conditions. Different non-uniformities.

Aspect	Makris et al. (DFE)	Axona
Skew dimension	Request rate per key (Zipf-distributed gets)	Crash failure / churn (nodes disappearing)
Trigger	Local threshold on RT or NR exceeded	Lookup failure / dead-peer discovery
Response	Migrate hot keys to underloaded peers	Anneal lowest-vitality synapse, reheat temperature
State change	Physical key migration + DFE redirect entries	Synaptome reweighting + new edges via LTP / triadic
Mechanism family	Cooperative location cache (DFE)	Cooperative location cache (hop caching)
Setting assumptions	24-node Redis cluster, full-mesh TCP, stable nodes, honest reporting	50K-node P2P overlay, partial mesh, churn, possibly adversarial
Pub/sub support	Out of scope (KV gets only)	First-class via axonal trees

Where they overlap. Both treat *self-monitoring + threshold-triggered protocol response* as the right control-loop architecture. Both use a "directory of exceptions" / "hop cache" — a cooperative override of the canonical placement function — as the data structure that implements the override. Both reduce a balance problem to a packing problem (FFD for them, axon-tree branching for us).

Where they differ. They assume **stable cooperative nodes with global load knowledge**; we cannot. Their FFD requires snapshot information our overlay can't cheaply collect. The *adaptation* needs translation: their migration becomes our *axon-tree root migration*, their FFD becomes our *load-aware AP scoring*.

The headline trade. *Their work targets content-popularity skew in stable clusters. Axona targets node-failure / churn skew in dynamic overlays. Bringing the two together — **load-aware AP scoring + threshold-triggered axon-root migration** — closes the gateway concentration failure mode the deck currently leaves open.*

Example Message Protocol

Reference slide — the wire-level protocol surface. Skim if you only care about results; consult when you want to know what Axona actually exchanges between peers.

Two layers. The application sees exactly two messages — SEND and RECEIVE. Everything else is the lower-level p2p wire protocol that implements them.

Application layer — what an application built on Axona actually calls:

Message	Purpose
SEND (target, payload)	Deliver a message toward a target — node, key, or topic
RECEIVE (handler)	Register to receive messages addressed here, or to a topic this node subscribes to

P2P wire layer — the messages Axona exchanges between peers to make SEND / RECEIVE work. Application code never constructs these directly.

Message	Purpose
PING / PONG	Liveness
FIND_NODE	DHT lookup step
ROUTE	Routed message toward a key (subscribe, publish, generic SEND)
SUBSCRIBE / UNSUBSCRIBE	Pub/sub membership; carries `lastSeenTs` for replay
PUBLISH	Publish to topic
DIRECT_DELIVER, ADOPT_SUBSCRIBERS, REPLAY_BATCH	Axonal pub/sub specifics

Common envelope (every wire message): version, type, sender, signature, timestamp, nonce, payload. Routing is **stateless in the protocol**. All state lives per-node: synaptome + axon roles.

Deployment considerations

- **Trust model.** No central authority. Public-key signatures authenticate peers.
- **Sybil resistance.** S2 cell prefix lightly discourages sybil swarms per-cell — not a full defense. Proof-of-location or a proof-of-work (**PoW**) join hurdle recommended for open deployments.
- **Provisioning.** Bootstrap via a small published sponsor set; fully decentralized thereafter.
- **Observability.** Nodes export local stats (synaptome health, LTP rate, role counts) for operator overlays.

The two-layer API — how Axona becomes real software

A working DHT has **two interfaces**, not one:

- The **DHT contract** — what the application sees: lookup, subscribe, publish, getMetrics, onEvent.
- The **Transport contract** — what the network exposes: openConnection, send, notify, onPeerDied, getLatency.

The **protocol** sits between them and depends on neither side directly.

```
Application → DHT contract → Axona protocol (NH-1) → Transport contract → Network
```

The simulator's `SimulatedNetwork` and the production `WebRTCTransport` both implement the **same** Transport contract — twelve methods, one signature each. **The protocol code does not know which it is talking to.** This is the property that lets the simulator be the deployment vehicle.

DHT contract — what the application sees

Eight verbs, organized by role:

Band	Methods
Lifecycle	`start`, `stop`, `join(sponsor)`, `leave`
Operations	`lookup(targetKey)`, `subscribe`, `unsubscribe`, `publish`
Identity & observability	`getNodeId`, `getSynaptome`, `getMetrics`, `onEvent`

Forbidden by design: no method that enumerates "all nodes", no method that takes a peer-id and returns that peer's state, no method that mutates the routing table.

A DHT instance owns *one node's view*. The simulator's Engine creates many DHT instances and orchestrates them — but that orchestration is not part of the contract.

Transport contract — what the network exposes

Twelve methods across four bands. Most interesting:

- `openConnection(peerId)` / `closeConnection(peerId)` — **synaptome admit / evict are channel open / close.** Bilateral cap enforced inside the transport: `openConnection` returns `false` if the remote refused.
- `send(peerId, type, payload)` for **request/response** (routing chain, two-hop probe).
- `notify(peerId, type, payload)` for **fire-and-forget** (LTP reinforce, hop caching, triadic introduction). No round-trip cost.
- `onPeerDied(handler)` + `getLatency(peerId)` — driven by a **1 Hz ping/pong heartbeat** on every open channel. Replaces the legacy god's-eye `nodeMap.get(p)?.alive` check with the same mechanism a real deployment uses.

Canonical pattern for parallel probes: `Promise.allSettled(peers.map(p => transport.send(...)))`. A slow or dead peer in one probe doesn't fail the whole batch.

Parity gate — same code path, both worlds

Years of Axona benchmark numbers transfer directly to production because **the protocol code is unchanged** between simulator and deployment. Twelve transport methods are the entire surface that swaps.

A 25 K-node parity-gate benchmark (post-refactor sim v0.70.22 vs pre-refactor v0.70.04 reference) confirmed every protocol within the 10 % target band:

Protocol	Hops Δ (global)	Latency Δ (global)
Kademlia	-2.8 %	-2.7 %
G-DHT	-0.3 %	+0.4 %
NX-17	+0.05 %	-0.2 %
NH-1	**+4.9 %**	**+1.3 %**

NH-1's small drift upward is the architecturally-honest cost: each peer makes its own next-hop decision from its own synaptome rather than a source-orchestrated walk reading across all intermediates. **It's the cost we pay in production anyway.**

Live deployment

The production-side of both contracts is now live. **Axona** is the protocol — and the name of the deployed network running it.

Contract	Production implementation	Live at
Transport	`axona-peer` — WebRTC data channels, 1 Hz heartbeat	https://axona.net
BootstrapService	`axona-bridge` — WebRTC offer/answer signaling	https://bridge.axona.net
DHT (Axona / NH-1)	`NeuromorphicDHTNH1` — unchanged from the simulator	runs inside `axona-peer`

`axona-peer` runs in the browser on Mac, Windows, Linux, iOS, and Android over real-world NATs (cellular, hotel WiFi, double-NAT). `axona-bridge` carries only the WebRTC offer/answer payloads between two peers — no application traffic — so the rendezvous role is interchangeable and a federated bridge mesh is on the Q4 2026 roadmap.

First end-user application. `civildefense.io` — a tap-to-report incident map. Anonymous P2P, geographic locality via the S2 prefix, 24-hour expiry from the replay-cache LRU. Built in weeks because the substrate primitives map directly.

Pub/sub application API — the five verbs.

Verb	Action
`publish`	author a new `SignedPost` into one of your topics
`subscribe`	attach to a publisher's topic; receive future posts
`pull`	fetch any post by content hash from its topic's relay tree
`reshare`	publish with a reference back to an upstream post
`metrics`	publisher-only — aggregate `delivery_count`, `pull_count`, `reshare_count`

`AxonPubSub.js` sits above `AxonManager`; encryption, schema, and ordering belong to the application. Cascade test at 2001 nodes verifies the counter invariants end-to-end.

Public repos: `github.com/axona-net/axona-peer` · `github.com/axona-net/axona-bridge` · `github.com/axona-net/dht-sim`.

Pub/sub failure modes we can name

Honest accounting of where the axonal tree shows seams:

- **Forwarder loss under churn.** When a tree-internal node dies mid-publish, its subtree is briefly unreachable until each member's next refresh. Today: each affected node's periodic re-subscribe routes to whichever live axon is now closest to `topicId` and re-attaches there — no explicit subtree move, the tree heals at the speed of the refresh interval. *Possible:* redundant forwarders or proactive health checks to shorten the gap.
- **Tree rebuild cost at scale.** O(S × F) for S subscribers and F forwarders. Negligible at 2 K subscribers; measurable at 50 K+. *Possible:* incremental updates rather than full rebuilds.
- **Gateway concentration.** A skewed synaptome can yield deep-narrow trees instead of broad-shallow ones, *and* a Zipf-popular topic concentrates load on its single root. Today: recursive delegation distributes load. *Possible:* secondary splitting on geographic cell when one gateway covers > 50 % of remaining children, plus **threshold-triggered hot-axon-root migration** with a DFE-style redirect entry (Makris et al. 2017) — the same mechanism Redis clusters use for hot-key skew.
- **Synaptome–tree coupling.** Annealing replacing a synapse that's also a forwarder leaves a stale tree until TTL. *Possible:* mark tree dirty on synapse eviction.
- **Byzantine resistance.** The system assumes honest nodes. *Possible:* MaxDisjoint replica placement (Harvesf & Blough 2007) for axon-tree roots, plus the Castro et al. (OSDI 2002) triplet — constrained routing tables, secure node-ID assignment, redundant routing — proof-of-location, cryptographic ID binding, reputation, multi-path verification.

Every item above is documented in Chapter 7 of the whitepaper with current mitigation and a candidate fix. None are blockers for the headline workloads in this deck — but each is a named, measurable failure mode the architecture should answer for in the next iteration.

Limitations and future directions

Known limitations

- **Warmup dependency.** Axona needs ~5 000 lookups before the synaptome converges. The first minute of deployment is suboptimal.
- **Synaptome capacity bounded at 50.** A deliberate cross-browser target. Server deployments could use 200–500.
- **Locality requires geographic IDs.** Without S2 prefix, regional locality collapses. *(Vivaldi-style RTT-only locality is future work.)*
- **Training is compute-optimizing, not path-shortening.** Cannot close the bootstrap → omniscient hop gap without changes to bootstrap or annealing.
- **Memory & bandwidth overhead.** ~50 synapses × ~80 B metadata ≈ 4 KB / node + axonal-tree state per subscribed topic. Modest, but worth measuring at 50 K+ subscribers.
- **Workload skew not yet stressed.** Today's pub/sub benchmark uses uniform topic activity. Real workloads are Zipf-distributed (few hot topics, long tail). Single-root axon trees may saturate under content popularity — methodology gap addressed in red-team Phase 2 (Zipf publish workload).
- **Churn rate is uniform in current benchmarks.** Real networks churn unevenly — corporate workdays, time-of-day peaks, regional outages. Variable-churn ("peak-hour") benchmarking is the sharper test of self-tuning; it is a methodology gap addressed in red-team Phase 2 (variable-churn benchmark, Ghinita-Teo style).

Future directions

- **RTT-driven locality (Vivaldi integration).** Today's locality is structural; an embedded coordinate system could drive locality without a geo-prefix.
- **Adaptive synaptome capacity.** Bump capacity during join-heavy periods; prune in steady state.
- **Global-pool annealing.** Periodically replace the lowest-vitality synapse with a *globally-sampled* candidate rather than a 2-hop sample — closes the bootstrap → omniscient gap.
- **Larger-scale evaluation.** 100 K / 250 K / 1 M nodes. Preliminary 50 K data exists; larger runs need server-side simulation.
- **Proof-of-location.** The S2 prefix is self-declared today. A verifiable location primitive would prevent prefix forgery.

Bandwidth distribution — the open question, now closed

*The single biggest unresolved deploy concern in the v0.3.38 red team. Issue #3 (bandwidth saturation, "Critical / Deploy blocker / 6–10 weeks") asked: does adaptive routing **amplify** hotspot formation (success-disaster oscillation) or **distribute** load? The simulator's "infinite bandwidth" assumption made it unfalsifiable — until we measured it.*

v0.70.04: per-node send/receive counters at every routing chokepoint. 16-run sweep × 4 protocols × 4 sizes (5K → 50K), then `annealRateScale` knee-search and 5%-churn comparison.

@ 50K nodes	Msgs Gini	Msgs max/mean	Nodes >100× mean
Kademlia	**0.940**	208×	**56**
G-DHT	**0.932**	213×	**62**
NH-1 default	0.661	47×	**0**
NH-1 throttled (rate=0.10)	0.79 *(at 25K)*	47×	**0**

The hypothesis the red team raised — *that adaptive routing concentrates more than static routing* — is empirically inverted. Kademlia/G-DHT develop 56–62 catastrophic-bandwidth nodes at 50K; **NH-1 has zero at any tested scale.**

The 25× volume tax was a knob, not a cost. `local_probe` *(the 2-hop annealing scan) is 89 % of NH-1 wire traffic.* `annealRateScale = 0.10` *cuts it 10× linearly, costs nothing in routing quality, and keeps every load-distribution metric strictly better than Kademlia.*

Bandwidth — the rate-knee and churn validation

`annealRateScale` rate-knee at 50K NH-1 (no churn, 10 sessions each):

rate	Total/cycle	local_probe	Gini	hot10×	vs default
0.05	63,847	18,224	0.866	852	6.5× cut
0.10	**82,650**	**36,989**	**0.834**	**575**	**5.0× cut**
0.25	137,363	92,453	0.770	735	3.0× cut
0.50	232,749	187,366	0.713	1,215	1.8× cut
1.00	414,929	369,552	0.661	1,428	(default)

Hops, time, success rate are **identical** across the entire range. The knee is at rate ≈ 0.10 — five-fold volume reduction with the lowest absolute hot-node count.

Churn validation @ 25K, 5 % per cycle:

Protocol	Total/cycle	Gini	hot10×	hot100×	Δ Gini vs no-churn
Kademlia	12,725	**0.919**	323	**7**	+0.006
G-DHT	15,817	**0.910**	321	**7**	+0.006
NH-1 default	356,358	0.654	198	0	+0.037
NH-1 throttled	**71,695**	**0.786**	364	**0**	**+0.012**

T_REHEAT still fires on dead-peer detection regardless of base rate, so churn-recovery is uncompromised by the throttle. Throttled NH-1 has the smallest Gini increase under churn of any protocol tested.

Bandwidth — what this changes in the red-team plan

Before (v0.3.38): Issue #3 was the only Tier 1 item where the *architectural property of the protocol* was unknown. Issues #1, #2, #4 were textbook engineering — known mitigations, bounded cost. Issue #3 was foundational: if the protocol amplifies concentration, no Phase 3 mitigation would help. The proposed Phase 3A-3E (per-node load reporting → load-aware AP scoring → hot-axon-root migration → MaxDisjoint replication → replay-cache load awareness) was 6–10 weeks of *new mechanism* gated on a hypothesis we couldn't yet test.

After (v0.70.04 sweeps, 16 + 11 + 15 = 42 runs):

- The foundational concentration question is **empirically resolved**: NH-1 distributes load broadly and stably; Kademlia/G-DHT concentrate it catastrophically at scale. The "amplifies hotspots" hypothesis is **falsified**.
- The 25× volume tax is **a single tunable parameter** (`annealRateScale`) with a clean knee (rate ≈ 0.10) and zero impact on routing quality.
- Phase 3 mechanisms (hot-axon-root migration, MaxDisjoint replication) are still relevant for the **Zipf-pub/sub case** — a single popular topic's root saturating from publish volume — but that is now a *bounded* sub-problem with an architectural fix already specified, not the framing of the entire load story.

Reclassification. *Issue #3 moves from "Tier 1 deploy blocker requiring net-new mechanism" to a tuning question with a measured answer. The red team's "brain is production-ready, body is not" verdict needs updating: the body is not as broken as feared. Transport-layer concerns (1, 2, 4) remain real engineering work; the architectural concern (3) is largely resolved.*

Key takeaways

1. **NH-1 collapses ~18 NX-17 rules into 12 rules organised under 5 operations.** A single vitality function — `weight × recency` — drives every admission decision.
2. **15 % server-class nodes (highway%) captures 70 % of the available latency improvement.** Realistic deployment doesn't need a uniform server fleet.
3. **Pub/sub on routed axonal trees: 100 % baseline, 100 % recovered delivery under 5 % churn.** No K-closest replication, no gossip, no parent tracking — just routed re-subscribe.
4. **The simulator catches its own bugs.** Skepticism — "if it looks too good to be true, it is" — has saved this deck from at least three retracted findings during this session.
5. **Locality in NH-1 is structural (S2 prefix), but the latency map is already in the routing decision.** AP scoring already factors observed RTT into next-hop choice; that latency map is a natural seed for a **Vivaldi-style coordinate system** that could replace the S2 prefix with a *learned* locality primitive — a forward direction we plan to explore in later iterations.
6. **Bandwidth concentration is empirically not a problem** *(NEW, v0.70.04).* Across 5K–50K nodes and under 5 % per-cycle churn, **NH-1 produces zero >100× hot nodes** at any tested scale. Kademlia and G-DHT produce 56–62 such nodes at 50K. The 25× volume tax of full annealing is a single tunable parameter (`annealRateScale`); at rate=0.10 NH-1 uses 5× the bandwidth of Kademlia — not 25× — with strictly better load distribution and identical routing quality.

Red team review

Three independent red-team passes, all deliberately adversarial — *what would break if we shipped this tomorrow?*

- `docs/red-team-analysis.md` (2026-04-21, NX-15 / NX-17 era) — transport friction, congestion, ID-space security. Most findings remain valid.
- `docs/red-team-analysis-nh1.md` (2026-04-29, NH-1 update) — tracks which gaps consolidation closed and which environmental "cheats" remain.
- `documents/NH1-RedTeam-v0.3.38.md` (2026-04-30, **13-issue priority list**) — composite ranking by occurrence × severity × detectability × time-to-fix × deploy-blocker status. Tiers the work into 4 deploy blockers, 4 correctness-under-stress items, and 5 operational-hardening items.

The combined verdict, in one paragraph:

> The simplification of NH-1 means the team is now perfectly positioned to tackle the environmental realities that were previously masked by NX-17's complexity. The "brain" of the DHT is production-ready. The body — connection setup, RPC timeouts, bandwidth saturation, jitter, asymmetric reachability, an adversarial ID space — is the next, *measurable, scopeable* problem.

Three sections follow:

- **Light** — what the architecture got right (all three reviews)
- **Dark** — environmental and structural gaps the simulator hides
- **Action plan** — three priority tiers (Deploy blockers · Correctness under stress · Operational hardening)

Total estimated work: *~6–8 weeks for two engineers to clear deploy blockers (Tier 1); 1–2 quarters for full coverage. Tier 2 and Tier 3 can be staged during early production.*

Red team — what the architecture got right (the Light)

1. LTP-driven routing affinity. Reinforcing edges based on *actual usage and measured latency* organically aligns the overlay with the physical network. Nodes build "highways" to reliable, fast peers rather than arbitrary XOR-close ones that may be on poor links. Both reviews flag this as the central architectural win — and it is what the 3δ floor result quantifies.

2. Unified admission gate (NX-17 → NH-1). NX-17 had stratified eviction floors, two-tier highway management, and adaptive decay all interacting. NH-1 replaces them with `_addByVitality` scored by `weight × recency`. ~10 % latency gap under heavy caps in exchange for a robust, predictable rule that prevents parameter over-fitting.

3. Publisher-prefix topic IDs. `topicId = publisher.cellPrefix(8b) ‖ hash₅₆(name)` deterministically pins each pub/sub root inside the publisher's S2 cell. Local traffic stays local; the publisher's synaptome is already LTP-trained for that exact region. Solves the "random-root geometry" problem that K-closest replication had.

4. Honest isolation of learning (geoBits = 0 ablation). Stripping the S2 prefix and re-running shows learning *alone* yields **+26 % (NX-17) / +8 % (NH-1) over Kademlia** — locality and adaptation separated by measurement, not assertion.

5. Documented failure modes. Forwarder loss under churn, tree rebuild cost, gateway concentration, synaptome-tree coupling, byzantine assumptions — every one named with current mitigation and candidate fix. The tree heals at the speed of the 10-second refresh, not magically via gossip.

Red team — remaining gaps (the Dark)

The **protocol** has improved. The **environment** it lives in has not. NH-1 over the real internet today would likely face cascading timeouts and congestion collapse before the routing logic ever gets to demonstrate itself.

1. Frictionless connection fantasy. Nodes use a peer for routing the moment they discover it. Real WebRTC requires ICE + STUN/TURN + DTLS — **1–3 s of blocking setup, two overlay round trips**. Slice World "unzips" elegantly because hundreds of new triadic closures cost nothing; in production, that would trigger hundreds of simultaneous handshakes and drop the bridge offline.

2. Asynchronous black holes & missing timeouts. RPCs to dead nodes return instantly because the simulator knows who is alive. **No timeouts, no dropped packets, no asymmetric path failures** (request goes through, reply doesn't). The 100 % pub/sub recovery under 5 % churn assumes instant detection. In the wild, every churn round costs multi-second timeout windows during which messages are lost.

3. Gateway concentration & infinite bandwidth. Hop cost is `10 ms + propagation` regardless of load. A highway node carrying 10 K pub/sub messages has the same modeled cost as an idle one. **Buffer saturation is invisible.** AP scoring keeps hammering the "best" nodes — risk of *success disaster*: LTP reinforces a node until it congests, abandons it, then flocks back when it recovers (oscillatory loops).

4. Jitter-free latency. Real RTTs fluctuate ±30 % from bufferbloat, asymmetric paths, and queueing. The simulator's distance-derived latency is monotone and clean. The EMA latency tracker has an easy job; high-frequency noise could prematurely decay good synapses or promote lucky-but-unstable ones.

5. Asymmetric reachability and bilateral assumption failure. The simulator assumes if A connects to B, B can reach A on the same edge. **Carrier-grade NAT, asymmetric firewalls, and asymmetric upstream/downstream bandwidth break that.** Every learning rule (LTP, hop caching, lateral spread, triadic closure) silently encodes the bilateral assumption — a one-directional bridge in Slice World heals only half the network; routing loops become possible when peers' views of each other diverge.

6. Sybil forgery & cell eclipse. The S2 prefix is self-declared. **An attacker can pick any prefix** and generate IDs that land in a target cell. The canonical mitigation is the **Castro et al. (OSDI 2002) triplet** — constrained routing tables + secure node-ID assignment + redundant routing — combined with route-diversity replica placement (Harvesf & Blough 2007). Proof-of-location, Vivaldi RTT clustering, and IP-ASN bounding are complementary and currently unimplemented.

Red team — Tier 1: deploy blockers

Issues 1–4 from the v0.3.38 ranking. ~18–30 person-weeks, partially parallelizable. The protocol's claimed properties cannot be verified in production until these are addressed.

1. Connection-setup model. Real WebRTC requires ICE + STUN/TURN + DTLS — **1.5–3 seconds blocking** per new synapse. Slice World "unzips" today only because hundreds of new triadic closures cost nothing.

- `CONNECTION_SETUP_MS = 1500–2000 ms` — new synapses sit in `PENDING`, excluded from AP scoring until setup elapses.
- **Browser-aware concurrency cap** — Chrome desktop 4 / Safari & mobile 2 ICE in flight; tab-backgrounding throttles to 1; GC-pause tolerance to suppress spurious timeout suspicion.

2. RPC timeouts and request/reply tracing. Today RPCs to dead nodes return instantly because the simulator knows. **No timeouts, no dropped packets, no asymmetric reply paths.**

- `RPC_TIMEOUT_MS = 3000 ms` — sends stall on silent failure before iterative-fallback / next AP hop.
- **Request/reply RPC refactor** — `routeMessage` traces forward and reverse paths. Either failure fails the RPC; reply may take a different path back.

3. Load-dependent AP scoring + bandwidth modeling. ~~Today hop cost is `10 ms + propagation` regardless of load. AP routing risks oscillatory *success disasters* — reinforce a node until it congests, abandon it, flock back when it recovers.~~ **Reclassified v0.70.04 — see "Bandwidth distribution" slides above.** Per-node send/receive counters at every chokepoint, 42-run sweep across protocols/sizes/churn/rate-knee. Result: **the architectural concentration hypothesis is falsified** — NH-1 distributes load (Gini 0.66 / zero >100× hot nodes at 50K) while Kademlia concentrates it (Gini 0.94 / 56 catastrophic nodes at 50K). The 25× volume tax is a tunable knob (`annealRateScale`), knee at 0.10. Tier-1 status removed from this issue.

- The **Zipf-pub/sub sub-case** (one popular topic root saturating from publish volume) remains relevant and will be addressed in Phase 3 *(see "Tier 2 / hot-axon-root migration & MaxDisjoint replication")*.

4. Jitter injection + LTP-EMA validation. Real RTTs fluctuate ±30 % from bufferbloat and queueing.

- Add `Normal(0, JITTER_SIGMA)` to per-hop RTT. Verify the LTP EMA doesn't oscillate; tune the EMA constant if it does.

Gate. *No deploy claim about latency, churn, or pub/sub holds until the simulator models all four sources of friction. Once it does, the existing measurements*

Red team — Tier 2: correctness under stress

Issues 5–8 from the v0.3.38 ranking. ~16–28 person-weeks. The protocol works in nominal conditions; under realistic adversarial or heterogeneous workloads it degrades. Staged rollout can manage this — but it must be measured, not assumed.

5. Sybil & cell eclipse hardening.

- **Vivaldi RTT integration** — replace self-declared S2 prefix with organically learned coordinates (Sybil-resistant locality without trusting peer self-claims).
- **Geographic proof-of-work / IP-ASN binding** — require geo-prefix to align with actual ASN/region; raises the cost of cell eclipse drastically.

6. Heterogeneous-churn convergence.

- **Variable-churn benchmark** — alternate 3/sec churn for 30 min with 0.5/sec for 90 min. Measures adaptation time at transitions.
- **Adaptive anneal driven by churn rate** *(Ghinita & Teo 2006).* Anneal cooling rate, reheat amount, staleness threshold all become functions of locally-observed `(μ, λ)`. Peers in stable regions anneal slowly; peers in high-churn regions anneal aggressively.
- **Liveness vs accuracy decoupled** *(Ghinita-Teo).* Independent threshold-triggered channels — one operator dial per channel.
- **Patchwork-style background liveness probes** *(Tapestry 2004).* Catches silent failures before routing errors fire.

7. Byzantine pub/sub mitigation.

- **MaxDisjoint topic replication** *(Harvesf & Blough 2007).* Replicate every axon-tree root at *d* disjoint locations. No single node — or single S2-cell eclipse — can silence a topic.
- **Zipf-distributed publish workload** + **hot-axon-root migration** *(Makris et al. 2017).* Threshold-triggered topic migration with DFE-style redirect at the original location, closing the *gateway concentration* failure mode under content popularity.

8. Asymmetric reachability and bilateral failure *(NEW).*

- **Per-direction `Synapse` fields** — `forwardReachable`, `reverseReachable`, `forwardLatencyEMA`, `reverseLatencyEMA`, `asymmetryFlag`. Updated on every successful exchange in either direction.
- **Direction-specific AP scoring** — exclude `forwardReachable=false` for forward routing; same for reverse during reply paths.
- **Loop detection in iterative fallback** — track visited node IDs; explicitly exclude already-visited on expansion.

Red team — Tier 3: operational hardening

Issues 9–13 from the v0.3.38 ranking. ~17–25 person-weeks. Long-tail items for production maturity, second-deployment confidence, and operator handoff.

9. Promoted-incoming bias and spam resistance *(NEW).* The "incoming promotion" rule has no defensive limit — a popular node's synaptome can become 5/45 outbound/incoming-promoted, losing diverse outbound coverage.

- **Track synapse origin** (`OUTBOUND_LEARNED | INCOMING_PROMOTED | BOOTSTRAP_SEED`).
- **Cap promoted-incoming at ~30 %** of synaptome budget; reduced inertia for promoted entries.
- **Per-source spam rate-limit** — sources connecting > 10/sec have promotion suppressed regardless of useCount.

10. Parameter sensitivity sweep + four-dial framework *(NEW).* The "12 parameters" claim only holds if individual parameters are robust and interactions are documented.

- **OFAT ±20 % / ±50 %** sweep per parameter at 25K nodes — identifies high-sensitivity parameters needing narrow operator bounds.
- **2D interaction sweep** across the 66 parameter pairs — surface joint-only effects.
- **Scale- and workload-aware defaults** — repeat at 1K/10K/50K and under uniform / Zipf / Slice-World workloads.

11. Replay-cache semantics under partition *(NEW).* The bounded replay cache is designed for transient churn, not multi-minute regional partitions.

- **Cache metadata on replay** — `(oldestTs, newestTs, capacity, gapsDetected)` so subscribers know whether they have a gap.
- **Multi-source replay reconciliation** — dovetails with MaxDisjoint replication; subscribers query several replicas in parallel and dedupe by `publishId`.
- **Partition-aware retention** — extend cache window when forward-attempt failure rate spikes.

12. Concurrent connection-setup contention *(NEW — folds into Tier-1 #1 once Tier 2 is in flight).* Browser concurrency limits cause learning rules to silently serialize. Adaptive throttling + GC-pause / tab-backgrounding handling already covered above; explicitly track here so the operability metric isn't lost.

13. Adversarial operator tuning + audit hygiene *(NEW).*

- **Multi-source metric verification** — dashboard churn rate cross-checked against locally-measured per-node samples; alert on disagreement.

References — DHT foundations

Whitepaper — `documents/Neuromorphic-DHT-Architecture.md` (this repository, v0.67)
Source + data — `github.com/YZ-social/dht-sim`

Architecture

- Saltzer, Reed, Clark · *End-to-End Arguments in System Design* (ACM TOCS 1984) — function placement principle
- Clark · *The Design Philosophy of the DARPA Internet Protocols* (SIGCOMM 1988)

Foundational DHT work

- Maymounkov & Mazières · ***Kademlia: A Peer-to-peer Information System Based on the XOR Metric*** (IPTPS 2002) — **the literal substrate of Axona**. XOR distance, K-buckets, α-parallel lookup, four-RPC protocol — all retained unchanged below the synaptome layer. NX-4 iterative fallback is unmodified Kademlia routing
- Rowstron & Druschel · ***Pastry: Scalable, decentralized object location and routing*** (Middleware 2001) — the canonical **three-tier routing state** (R / L / M) and **locality-preserving join**; substrate for SCRIBE and PAST. Axona's synaptome consolidates Pastry's three tables into a single vitality-scored set
- Zhao, Huang, Stribling, Rhea, Joseph, Kubiatowicz · ***Tapestry: A Resilient Global-Scale Overlay for Service Deployment*** (IEEE JSAC 2004) — the closest historical ancestor of Axona's combined locality + soft-state + resilience architecture; substrate for OceanStore, Bayeux multicast, and Mnemosyne

References — Pub/sub and latency

Pub/sub infrastructure

- Castro, Druschel, Kermarrec, Rowstron · ***SCRIBE: A Large-Scale and Decentralized Application-Level Multicast Infrastructure*** (IEEE JSAC 2002) — routed subscribe + **reverse-path forwarding multicast tree** layered on Pastry; **the structural ancestor of Axona's axonal-tree pub/sub**
- Zhuang, Zhao, Joseph, Katz, Kubiatowicz · *Bayeux: An architecture for scalable and fault-tolerant wide-area data dissemination* (NOSSDAV 2001) — the analogous multicast layer on Tapestry

Latency-aware DHTs

- Dabek, Li, Sit, Robertson, Kaashoek, Morris · ***Designing a DHT for low latency and high throughput*** (NSDI 2004) — DHash++; the **3δ floor** analysis (§ 4.3) anchors our absolute-latency reference
- Freedman, Mazières · *Sloppy Hashing and Self-Organizing Clusters* (IPTPS 2003) — the Coral DSHT design
- Freedman, Freudenthal, Mazières · ***Democratizing Content Publication with Coral*** (NSDI 2004) — Coral CDN deployment

References — Adaptive systems

Adaptive coordinates

- Dabek, Cox, Kaashoek, Morris · ***Vivaldi: A Decentralized Network Coordinate System*** (SIGCOMM 2004) — synthetic-coordinate primitive; candidate replacement for the self-declared S2 prefix
- Cox, Dabek, Kaashoek, Li, Morris · *Practical, Distributed Network Coordinates* (HotNets 2003) — earlier coordinate-system design
- Ledlie, Gardner, Seltzer · *Network Coordinates in the Wild* (NSDI 2007) — Vivaldi behavior in deployment

Adaptive maintenance / churn handling

- Mahajan, Castro, Rowstron · *Controlling the Cost of Reliability in Peer-to-peer Overlays* (IPTPS 2003) — early adaptive-maintenance methodology; cost-of-reliability framework
- Krishnamurthy, El-Ansary, Aurell, Haridi · *A statistical theory of Chord under churn* (IPTPS 2005) — analytical model of Chord routing-table accuracy under Poisson churn
- Ghinita, Teo · ***An Adaptive Stabilization Framework for Distributed Hash Tables*** (IPDPS 2006) — local statistical estimation of `(μ, λ, N)` + threshold-triggered liveness / accuracy checks; closest match in *mechanism family* to the Neuromorphic self-tuning instinct

References — Production resilience

Byzantine resistance / route diversity

- Castro, Druschel, Ganesh, Rowstron, Wallach · ***Secure Routing for Structured Peer-to-Peer Overlay Networks*** (OSDI 2002) — the foundational Byzantine-DHT paper; the **constrained routing + secure ID assignment + redundant routing** triplet
- Harvesf, Blough · ***The Design and Evaluation of Techniques for Route Diversity in Distributed Hash Tables*** (IEEE P2P 2007) — **MaxDisjoint replica placement** + Neighbor Set Routing; 90 % lookup success at 50 % node failure

Load balancing / hotspot mitigation

- Karger, Lehman, Leighton, Panigrahy, Levine, Lewin · *Consistent Hashing and Random Trees* (STOC 1997) — the foundational consistent-hashing paper; load-uniformity guarantees under uniform request rates
- Rao, Lakshminarayanan, Surana, Karp, Stoica · *Load balancing in structured P2P systems* (IPTPS 2003) — virtual servers + many-to-one / many-to-many migration schemes
- Makris, Tserpes, Anagnostopoulos · ***A novel object placement protocol for minimizing the average response time of get operations in distributed key-value stores*** (IEEE BIGDATA 2017) — **Directory For Exceptions (DFE)** + threshold-triggered migration; 86 % response-time reduction on the hotspot

References — substrate

The learning, decay, pruning, and topology priors that Axona translates from biology and classical CS into routing-table maintenance.

Substrate (learning, decay, pruning)

- Hebb · *The Organization of Behavior* (1949) — synaptic potentiation
- Ebbinghaus · *Über das Gedächtnis* (1885) — exponential forgetting curve
- Frey & Morris · *Synaptic tagging and the late phase of LTP* (Nature 1997) — biological analog of weight × recency retention
- LeCun, Denker, Solla · *Optimal Brain Damage* (NeurIPS 1989) — prune lowest-magnitude connections
- O'Neil, O'Neil, Weikum · *The LRU-K Page Replacement Algorithm* (SIGMOD 1993) — multi-history recency for cache replacement
- Watts & Strogatz · *Collective Dynamics of Small-World Networks* (Nature 1998)
- Google · *S2 Geometry Library* (2011) — https://s2geometry.io/